

Lab 4 - VibeCoding: Mastermind - Complete LLM chat history

Assistant: Claude (Claude Code, model claude-sonnet-5-5). Exported from the Claude Code desktop session "Lab 4 Mastermind implementation" with the session-export feature, then rendered to PDF by a script.
Contents: 2 user messages, 22 assistant messages, 65 tool calls with their results, in order.
Omissions (rendering only): screenshots/images are shown as placeholders (12); tool outputs longer than 8000 characters are truncated (1); the assistant's internal reasoning blocks are not rendered; harness-injected <system-reminder> blocks (0) are removed, since they carry account metadata, not conversation.
Conversation link: [claude://claude.ai/epitaxy/local_804df02a-218d-4259-970a-fa0ebdf64c7a](https://claude.ai/epitaxy/local_804df02a-218d-4259-970a-fa0ebdf64c7a) (local app deep link, see LLM_Chat_Link.txt)

USER 2026-10-07 16:46:14Z

<pasted_content id="5177">

You are acting as my autonomous Software Engineering Lab 4 implementation agent.
Your job is to complete the entire Lab 4 assignment end-to-end on my computer, including inspecting the assigned repository, reproducing the original bug, recording the required before/after gameplay videos, implementing every required task, testing everything, maintaining clean Git history with separate commits, exporting the complete LLM chat history, organizing all submission artifacts, and pushing the completed Lab 4 into my personal GitHub repository. Do not merely tell me what to do. Actually perform the work using the available terminal, filesystem, browser, screen-control, and recording capabilities.
Only ask me to intervene when there is a genuinely unavoidable blocker such as GitHub authentication, OS security permission, or a tool capability that is unavailable. Do not ask for confirmation for normal implementation decisions.

1. OFFICIAL SOURCES / REPOSITORIES

Assigned repository:

https://github.com/SETAPESU26/20_mastermind
My personal Software Engineering lab repository:
<https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569>
The Lab 4 student handout is the authoritative assignment context.
The assigned repository is Scenario 20 – Mastermind.
Existing assigned project files:

- * main.py – entry point
- * game.py – game loop and history
- * logic.py – exact/partial match calculation
- * requirements.txt – dependency declaration

The assigned README must be treated as the source of truth for the actual Mastermind requirements.
Do not modify, delete, rewrite, or damage Labs 1-3 in my personal repository.
My personal repo already uses top-level lab folders such as:

Lab1/
Lab2/
Lab3/

Therefore create:

Lab4/
and keep the entire Lab 4 submission self-contained there.

2. NON-NEGOTIABLE ASSIGNMENT RULES

Follow these rules strictly:

1. This is an individual assignment.
2. Inspect the original project before asking for/implementing changes.
3. Reproduce the original defect before modifying the source.
4. Record a real 10-second BEFORE gameplay video showing the broken behaviour.
5. Implement Task 1.
6. Test Task 1.
7. Create a separate Git commit for Task 1.
8. Implement Task 2.
9. Test Task 2.
10. Create a separate Git commit for Task 2.
11. Implement Task 3.
12. Test Task 3.
13. Create a separate Git commit for Task 3.
14. Implement Task 4.
15. Test Task 4.
16. Create a separate Git commit for Task 4.
17. Record a real 10-second AFTER gameplay video showing the fixed bug and newly added features.
18. Preserve the complete LLM conversation/chat history.
19. Export the complete LLM chat history to a PDF if the environment allows it.
20. Preserve/save the Claude conversation/share link as well.
21. Put the required deliverables under Lab4/.
22. Push the final work to my personal repository:
<https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569>
23. Do NOT create a PR against:
https://github.com/SETAPESU26/20_mastermind
24. Do NOT push anything to the upstream SETAPESU26 repository.
25. Do not add unnecessary external dependencies.
26. Do not add CSV, JSON, SQLite, databases, or any other persistence.
27. Keep all game state in memory.
28. Do not replace the project with an unrelated implementation.

29. Keep the implementation understandable, modular, and faithful to the original codebase.
30. Do not fabricate test results, recordings, screenshots, commits, or chat history.
31. Verify every deliverable actually exists before pushing.

=====

3. FIRST: INSPECT EVERYTHING

Before changing anything:

A. Clone or otherwise obtain a clean copy of the assigned repository:

https://github.com/SETAPESU26/20_mastermind

B. Inspect:

- * README.md
- * main.py
- * game.py
- * logic.py
- * requirements.txt

C. Inspect my personal repository:

<https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569>

D. Check the existing folder structure and Git history.

E. Do not alter Labs 1-3.

F. Establish a clean working structure for Lab4.

G. Run the original untouched Mastermind code.

H. Understand precisely how the original defect occurs before making any source changes.

4. ORIGINAL DEFECT TO REPRODUCE

The original logic contains duplicate-symbol handling that can count the same occurrence multiple times.

The current implementation effectively does:

exact = number of exact-position matches

partial = number of guessed symbols that exist anywhere in the code - exact

This is incorrect when symbols are repeated because the same code occurrence can contribute repeatedly.

A deterministic example that should be used to reproduce the original defect is:

code:

4413

guess:

4444

With the original logic:

exact = 2

and the broken implementation can report:

partial = 2

even though the two 4s in the code have already been consumed by the two exact matches.

The correct result is:

exact = 2

partial = 0

Use a deterministic runtime seed or another non-invasive runtime technique to make the BEFORE recording reproducible, but DO NOT modify the original source code before the before-video is recorded.

The baseline recording must genuinely demonstrate the original defect.

5. BEFORE VIDEO

Before changing the application source code:

1. Launch the untouched original application.
2. Reproduce the duplicate-symbol bug.
3. Record approximately/exactly 10 seconds of actual screen footage.
4. The recording should visibly show:
 - * the Mastermind application running,
 - * the relevant input,
 - * the incorrect feedback,
 - * enough context to make the broken behaviour obvious.
5. Do not fake or synthesize the video.
6. Prefer a clean terminal/application window with no unrelated personal information visible.
7. Verify the resulting file duration with ffprobe or an equivalent tool.
8. Save it temporarily until the final Lab4 packaging stage.

Target filename:

before_10s.mp4

If the raw recording is unnecessarily large, compress it to a reasonable GitHub-friendly H.264 MP4 while preserving approximately 10 seconds and readable terminal text.

6. TASK 1 – DUPLICATE-AWARE FEEDBACK

Implement the assigned README Task 1.

Requirement:

Each code position can be counted at most once.

Exact matches must be resolved before partial matches.

The implementation must correctly handle repeated symbols.

Before finalizing Task 1, test at minimum:

1. all exact matches
2. repeated symbols
3. no matches
4. mixed exact/partial matches
5. asymmetric duplicate cases
6. repeated guesses against repeated codes

Prefer a clean two-pass or equivalent duplicate-aware algorithm.

Do not unnecessarily restructure the whole project.
After implementation:

- * run the tests,
- * manually verify representative cases,
- * inspect the diff,
- * confirm there are no unrelated changes.

Then create a dedicated commit:

Lab 4: Fix duplicate-aware feedback

Do not combine Task 1 with later tasks in this commit.

=====

7. TASK 2 – COMPLETE GAME LIFECYCLE

Implement the assigned README Task 2.

Requirements:

- * clear win handling,
- * clear loss handling,
- * preserved guess history,
- * enforced guess limit,
- * no state changes after the game ends.

The game must correctly terminate after:

- * a successful win,
- * exhausting the allowed guesses,
- * quitting.

Verify that once the game ends:

- * no new guesses are accepted,
- * history does not change,
- * turns do not change,
- * no additional feedback is processed.

Test:

1. immediate win,
2. win after multiple guesses,
3. final-turn win,
4. loss after all allowed guesses,
5. quitting,
6. state remains unchanged after termination,
7. history remains intact.

Run the full relevant tests.

Review the diff.

Create a separate commit:

Lab 4: Complete game lifecycle

Do not combine this with Tasks 3 or 4.

=====

8. TASK 3 – DIFFICULTY MODES

Implement the assigned README Task 3.

Add difficulty choices that vary:

- * code length,
- * symbol range,
- * allowed guesses.

The feedback algorithm must continue working for every supported configuration.

Choose sensible, clearly distinguishable difficulty modes while staying faithful to the assignment.

Every supported mode must have:

- * a different/meaningful configuration,
- * valid code generation,
- * valid guess validation,
- * the correct guess limit,
- * correct feedback,
- * working win/loss logic.

Do not introduce unnecessary packages.

Keep configuration modular and understandable.

Test every difficulty mode.

Also verify duplicate-aware feedback still works across all modes.

Then create a dedicated commit:

Lab 4: Add difficulty modes

Do not combine it with Task 4.

=====

9. TASK 4 – HISTORY AND INPUT ROBUSTNESS

Implement the assigned README Task 4.

Requirements:

- * readable guess history,
- * malformed guesses rejected,
- * invalid guesses do NOT consume turns,

- * each feedback result corresponds to exactly one accepted guess.

Test malformed input including appropriate cases such as:

- * wrong length,
- * invalid symbols,
- * empty input where applicable,
- * malformed values,
- * quit command,
- * repeated invalid guesses.

Verify invalid input:

- * does not append to history,
- * does not decrease remaining turns,
- * does not generate feedback,
- * does not change game state.

Verify valid input:

- * is recorded exactly once,
- * decrements turns exactly once,
- * produces exactly one feedback result.

The history shown to the user should be easy to read.

Then create a dedicated commit:

Lab 4: Harden input and history

=====

10. REQUIRED TEST MATRIX

Before finalizing the project, comprehensively test all of the following:

Functional:

- * exact matches,
- * repeated symbols,
- * no matches,
- * mixed exact and partial matches,
- * every difficulty,
- * invalid guesses,
- * malformed guesses,
- * history,
- * final-turn win,
- * loss,
- * quitting.

Boundary/state cases:

- * first-turn win,
- * last-turn win,
- * loss on final allowed guess,
- * invalid input immediately before a winning guess,
- * multiple invalid inputs,
- * repeated symbols in both code and guess,
- * game state after game termination.

Make the tests reproducible.

Use Python standard-library testing facilities where appropriate.

Do not add external testing frameworks unless absolutely required; preferably use unittest or a small internal test suite.

Run the complete test suite from a clean state.

Do not claim tests pass until they actually pass.

=====

11. QUALITY CHECK

After Tasks 1-4 are implemented:

Inspect the entire project for:

- * syntax errors,
- * runtime errors,
- * incorrect edge cases,
- * duplicated logic,
- * unnecessary complexity,
- * dead code,
- * unnecessary dependencies,
- * accidental persistence,
- * accidental changes outside Lab4,
- * poor naming,
- * unclear flow.

Run:

- * the normal application,
- * the full automated tests,
- * representative manual gameplay.

Ensure the original application concept remains Mastermind and not a rewritten unrelated project.

=====

12. AFTER VIDEO

Once the final implementation is working:

Create a REAL 10-second gameplay recording.

The AFTER video should demonstrate as much of the following as possible within approximately 10 seconds:

- * launching the improved game,
- * selecting a difficulty,
- * invalid input being rejected without consuming a turn,
- * a valid guess,
- * correct duplicate-aware feedback,
- * readable history,
- * successful win OR clearly demonstrated lifecycle behaviour.

Optimize the interaction so the evidence is visible and understandable within 10 seconds.

Use deterministic runtime seeding or another safe non-source-changing technique for the recording if necessary so the desired demonstration is repeatable.

Do not modify production logic merely to make the video easier.

The recording must show the actual final application.

Target filename:

after_10s.mp4

Verify the file duration.

Compress if necessary while preserving readability.

13. SUBMISSION FOLDER STRUCTURE

Create this clean structure inside my personal repo:

```
Lab4/
├── README.md
├── mastermind/
│   ├── README.md
│   ├── main.py
│   ├── game.py
│   ├── logic.py
│   ├── requirements.txt
│   ├── tests/
│   └── test_mastermind.py
├── evidence/
│   ├── before_10s.mp4
│   ├── after_10s.mp4
│   ├── LLM_Chat_History.pdf
│   └── LLM_Chat_Link.txt
└── ...
```

Only add additional files when genuinely useful and justified.

Do not include temporary files, caches, virtual environments, build artifacts, random screenshots, or unrelated generated files.

14. LAB4 README

Create a professional Lab4/README.md.

It should include:

Lab 4 – VibeCoding: Mastermind

Course/lab identification.

Assigned repository:

https://github.com/SETAPESU26/20_mastermind

Personal submission repository:

<https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569>

Objective

Briefly explain that the lab fixes the original Mastermind defect and implements Tasks 1-4 using an LLM coding assistant.

Tasks Completed

Document:

- * Task 1 – duplicate-aware feedback
- * Task 2 – complete game lifecycle
- * Task 3 – difficulty modes
- * Task 4 – history and input robustness

Testing

Provide a concise test matrix showing that all required edge cases were tested.

Deliverables

Link to:

- * before_10s.mp4
- * after_10s.mp4
- * LLM_Chat_History.pdf
- * LLM_Chat_Link.txt
- * updated source code

Running the Game

Give the correct command.

Constraints Verified

Explicitly state:

- * no unnecessary external dependencies,
- * no persistence,
- * all state kept in memory,
- * original project structure/concept retained.

Git Commits

List the separate commits for Tasks 1-4.

LLM Usage

State that Claude was used as a coding assistant and that the complete chat history is included.

Do not invent claims.

15. ORIGINAL README PRESERVATION

Inside:

Lab4/mastermind/

preserve the assigned project's README information.

Do not silently replace it with a completely different project README.

A small implementation/testing section may be added if useful, but preserve the original problem statement and requirements.

16. CHAT HISTORY DELIVERABLE

The assignment explicitly requires the complete LLM chat history.

At the end of the work:

1. Preserve the entire Claude conversation used for this lab.
2. Do NOT submit merely a summary.
3. Include the prompts used for Task 1, Task 2, Task 3, and Task 4.
4. Include Claude's responses/changes/testing discussion.
5. Export the full visible conversation to PDF if supported.
6. Save it as:

Lab4/evidence/LLM_Chat_History.pdf

7. Also create:

Lab4/evidence/LLM_Chat_Link.txt

containing the actual Claude share/conversation page link.

If a direct share link is unavailable, use the best legitimate conversation/export mechanism available and document exactly what was generated.

Do not fabricate a URL.

Verify that the PDF actually contains the complete conversation.

17. GIT WORKFLOW

Use clean Git history.

First establish the Lab4 baseline in my personal repository.

Then use separate commits for each task:

Commit 1:

Lab 4: Add Mastermind baseline

Commit 2:

Lab 4: Fix duplicate-aware feedback

Commit 3:

Lab 4: Complete game lifecycle

Commit 4:

Lab 4: Add difficulty modes

Commit 5:

Lab 4: Harden input and history

Then add the final evidence/documentation package in a separate final commit, for example:

Lab 4: Add submission evidence and documentation

Do not squash the task commits.

Before every commit:

- * run tests,
- * inspect git diff,
- * make sure only relevant work is included.

At the end:

git status

must be clean.

Verify:

git log --oneline

shows the separate Task 1-4 commits.

18. GITHUB SAFETY

Your destination is:

<https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569>

The upstream assigned repository is:

https://github.com/SETAPESU26/20_mastermind

Never push changes to the upstream repository.

Never open a PR against the upstream repository.

Push only to my personal repository.

Do not accidentally overwrite Labs 1-3.

Before pushing, inspect:

git remote -v

and verify which remote is writable.

Then push the final Lab4 work to my personal repo.

After pushing, verify the actual GitHub repository online and ensure:

Lab4/

exists,

all expected files are present,

the commit history is visible,

and no previous lab was damaged.

```

=====
19. OPTIONAL ROOT README UPDATE
My personal repository currently lists the completed labs.
After everything else is complete, update the root README only if needed to add a clean Lab 4 entry pointing to:
Lab4/
Do not make unrelated edits.
Use a separate documentation commit if this is necessary.
=====
20. FINAL VALIDATION CHECKLIST
Do not finish until ALL of these are true:
[ ] Original assigned repository inspected.
[ ] Original defect reproduced.
[ ] BEFORE video exists.
[ ] BEFORE video is approximately 10 seconds.
[ ] BEFORE video clearly demonstrates the bug.
[ ] Task 1 implemented.
[ ] Task 1 tested.
[ ] Task 1 has its own commit.
[ ] Task 2 implemented.
[ ] Task 2 tested.
[ ] Task 2 has its own commit.
[ ] Task 3 implemented.
[ ] Task 3 tested.
[ ] Task 3 has its own commit.
[ ] Task 4 implemented.
[ ] Task 4 tested.
[ ] Task 4 has its own commit.
[ ] Full required test matrix passes.
[ ] AFTER video exists.
[ ] AFTER video is approximately 10 seconds.
[ ] AFTER video shows the improved game.
[ ] Complete LLM chat history exported.
[ ] Chat/share link saved.
[ ] Lab4 README created.
[ ] Updated code included.
[ ] No persistent storage.
[ ] No unnecessary dependencies.
[ ] No unrelated project rewrite.
[ ] Labs 1-3 untouched.
[ ] No PR created against SETAPESU26.
[ ] Final Git history contains separate Task 1-4 commits.
[ ] Working tree clean.
[ ] Changes pushed to my personal GitHub repository.
[ ] Final GitHub repository checked online.
[ ] All required evidence is actually accessible.
=====
21. EXECUTION STYLE
Work autonomously and continuously.
Do not stop after explaining the solution.
Do not just generate code in chat.
Actually:

* inspect,
* reproduce,
* record,
* implement,
* test,
* commit,
* record again,
* export evidence,
* package,
* push,
* verify.

Keep the implementation minimal, clean, modular, and faithful to the original repository.
Use the LLM interaction itself as a proper VibeCoding lab workflow: inspect first, make focused task-specific
requests, explain changes when useful, test them, and verify the result.
Do NOT replace the whole project with a new unrelated implementation.
When you have genuinely completed everything, give me a final concise report containing:

1. final GitHub Lab4 URL,
2. final commit list,
3. exact paths of the two videos,
4. exact path of the chat history PDF,
5. exact path of the chat link file,
6. test result summary,
7. confirmation that Labs 1-3 were not modified.

Do not report success unless you have actually verified it.
</pasted_content id="5177">

CLAUDE 2026-10-07 16:46:16Z
I'll start by checking the tooling and getting both repositories.

TOOL CALL 2026-10-07 16:46:17Z

```

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4" && ls -la && which git gh ffmpeg ffprobe python3 screencapture asciinema; python3
--version; git --version; gh auth status 2>&1 | head -5; git config --global user.name; git config --global user.email
```

TOOL RESULT 2026-10-07 16:46:19Z

```
total 144
drwxr-xr-x@ 3 aadi  staff    96 Oct  7 22:09 .
drwx-----@ 112 aadi  staff   3584 Oct  7 22:09 ..
-rw-r--r--@ 1 aadi  staff  71595 Oct  7 10:49 Lab_4_VibeCoding_Student_handout.pdf
/opt/homebrew/bin/git
/opt/homebrew/bin/gh
ffmpeg not found
ffprobe not found
/opt/homebrew/bin/python3
/usr/sbin/screencapture
asciinema not found
Python 3.14.0
git version 2.54.0
github.com
? Logged in to github.com account Sonuaadi0706 (keyring)
- Active account: true
- Git operations protocol: https
- Token: gho_*****
Sonuaadi0706
sonuaadish0706as@gmail.com
```

TOOL CALL 2026-10-07 16:46:22Z

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4" && git clone https://github.com/SETAPESU26/20_mastermind.git
/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream 2>&1 |
tail -2; git clone https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569.git personal 2>&1 | tail -2; ls personal; cd
personal && git log --oneline | head; git remote -v; cat README.md
```

TOOL CALL 2026-10-07 16:46:22Z

```
[Read]
{
  "file_path": "/Users/aadi/Desktop/SE lab 4/Lab_4_VibeCoding_Student_handout.pdf",
  "pages": "1-6"
}
```

TOOL RESULT 2026-10-07 16:46:28Z

```
Cloning into
'/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream'...
Cloning into 'personal'...
Lab1
Lab2
Lab3
README.md
46563d3 Add Lab 3: Component Modelling & Architectural Pattern Selection
edbf4b Add Lab 2: Agile Backlog Creation & Sprint Simulation in Jira
e6a4502 Add Lab 1: Requirements Engineering & UML Use-Case Modelling (Problem Statement #46)
f40f2f1 Initial commit
origin https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569.git (fetch)
origin https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569.git (push)
# SE-Labs-PES1UG24CS569

Software Engineering lab submissions for **PES1UG24CS569** (PES University, Dept. of CSE).
Each lab lives in its own top-level folder with its own deliverables and README.

## Labs

| Lab | Topic | Folder |
|---|---|---|
| Lab 1 | Requirements Engineering & UML Use-Case Modelling - *Remote Team Time-Tracking & Project Approver* (Problem Statement #46) | [ `Lab1/` ](Lab1/) |
| Lab 2 | Agile Backlog Creation & Sprint Simulation in Jira - Epics, User Stories, story points, two sprints & burndown charts | [ `Lab2/` ](Lab2/) |
| Lab 3 | Component Modelling & Architectural Pattern Selection - evaluated Layered/Microservices/Client-Server, selected Microservices, and produced a UML Component Diagram (8 components, 9 interfaces) with a written justification | [ `Lab3/` ](Lab3/) |

See each lab's folder for its own README with the full problem statement, approach, and a description of every deliverable inside it.
```

TOOL RESULT 2026-10-07 16:46:28Z

```
PDF pages extracted: 1 page(s) from /Users/aadi/Desktop/SE lab 4/Lab_4_VibeCoding_Student_handout.pdf (69.9KB)
[image omitted]
```

TOOL CALL 2026-10-07 16:46:31Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream &&
ls -la && git log --oneline && for f in README.md main.py game.py logic.py requirements.txt; do echo "=== $f"; cat -A
$f | head -0; cat $f; done
```

TOOL RESULT 2026-10-07 16:46:32Z

```
total 40
drwxr-xr-x@ 8 aadi  wheel    256 Oct  7 22:16 .
drwx-----@ 3 aadi  wheel     96 Oct  7 22:16 ..
drwxr-xr-x@ 12 aadi  wheel    384 Oct  7 22:16 .git
-rw-r--r--@ 1 aadi  wheel   2915 Oct  7 22:16 README.md
-rw-r--r--@ 1 aadi  wheel    960 Oct  7 22:16 game.py
```

```

-rw-r--r--@ 1 aadi wheel 236 Oct 7 22:16 logic.py
-rw-r--r--@ 1 aadi wheel 79 Oct 7 22:16 main.py
-rw-r--r--@ 1 aadi wheel 109 Oct 7 22:16 requirements.txt
63e3c15 Updated scenario title
5486f20 Initial commit
=== README.md
cat: illegal option -- A
usage: cat [-belnstuv] [file ...]
head: illegal line count -- 0
# Scenario 20 - Mastermind

A terminal code-breaking game with repeated symbols and structured feedback.

## Provided files

- `main.py` - entry point.
- `game.py` - game loop and history.
- `logic.py` - exact and partial match calculation.
- `requirements.txt` - dependency declaration.

## Setup

```bash
python main.py
```

## Before changing the code

Construct codes and guesses containing repeated symbols. Reason about how many times a
single code occurrence should be allowed to contribute to feedback.

## Task 1 - Correct duplicate-aware feedback

Implement feedback so each code position can be counted at most once. Exact matches
must be resolved before partial matches.

**Done when:** repeated-symbol cases produce the correct exact and partial counts.

## Task 2 - Complete game lifecycle

Add clear win/loss handling, preserve guess history, enforce the guess limit, and
prevent state changes after the game ends.

## Task 3 - Difficulty modes

Add difficulty choices that vary code length, symbol range, and allowed guesses.
The feedback algorithm must work for all supported configurations.

## Task 4 - History and input robustness

Present a readable guess history and reject malformed guesses without consuming turns.
Feedback must correspond to one accepted guess only.

## Required testing

Test exact matches, repeated symbols, no matches, mixed exact/partial cases, every
difficulty, invalid guesses, final-turn wins, losses, and quitting.

## LLM usage

You may use an LLM during the lab. The goal is to use it as a coding assistant while
retaining responsibility for understanding and testing the result.

- Inspect the existing code before asking for changes.
- Ask for explanations when you do not understand a proposed change.
- Test generated code against the stated behaviour and edge cases.
- Keep your complete LLM chat history for submission.
- Do not replace the whole project with an unrelated implementation.
- Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.

## Submission checklist

- [ ] Task 1 completed and the original defect was reproduced and fixed.
- [ ] Tasks 2-4 completed and tested.
- [ ] Boundary and invalid-input cases tested.
- [ ] No unnecessary external dependencies added.
- [ ] No persistent storage added.
- [ ] Code remains understandable and modular.
- [ ] Complete LLM chat-history link included.

## Folder structure

```text
scenario-08-mastermind/
├── README.md
├── requirements.txt
├── main.py
├── game.py
└── logic.py
```

## Submission Checklist

Submission is only the following three things:

```

```

- [ ] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [ ] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [ ] The Chat/LLM used page link, with the complete chat history
=== main.py
cat: illegal option -- A
usage: cat [-belnstuv] [file ...]
head: illegal line count -- 0
from game import Mastermind

if __name__ == "__main__":
    Mastermind().run()
=== game.py
cat: illegal option -- A
usage: cat [-belnstuv] [file ...]
head: illegal line count -- 0
import random
from logic import feedback

class Mastermind:
    def __init__(self):
        self.code = [str(random.randint(1, 6)) for _ in range(4)]
        self.history = []
        self.turns = 10

    def run(self):
        print("Mastermind - enter four digits from 1 to 6.")
        while self.turns:
            raw = input(f"{self.turns} turns left > ").strip()
            if raw.lower() == "q":
                return
            if len(raw) != 4 or any(ch not in "123456" for ch in raw):
                print("Enter exactly four digits from 1 to 6.")
                continue
            guess = list(raw)
            exact, partial = feedback(self.code, guess)
            self.history.append((raw, exact, partial))
            self.turns -= 1
            print("Exact:", exact, " Partial:", partial)
            if exact == 4:
                print("Cracked the code!")
                return
        print("The code was", "".join(self.code))
=== logic.py
cat: illegal option -- A
usage: cat [-belnstuv] [file ...]
head: illegal line count -- 0
def feedback(code, guess):

    # the code, so duplicate symbols can consume the same code occurrence.
    exact = sum(a == b for a, b in zip(code, guess))
    partial = sum(ch in code for ch in guess) - exact
    return exact, partial
=== requirements.txt
cat: illegal option -- A
usage: cat [-belnstuv] [file ...]
head: illegal line count -- 0
# Standard-library-only scenario.
# No third-party packages are required.
# Recommended Python version: 3.9+
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4

```

TOOL CALL 2026-10-07 16:46:49Z

```

[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && ls; which
avconvert ainfo mdls osascript expect brew; python3 -c "import imageio_ffmpeg" 2>&1 | tail -1; screencapture -x
t.png; ls -la t.png; screencapture -v -V 2 t.mov 2>&1; ls -la t.mov; mdls -name kMDItemDurationSeconds t.mov

```

TOOL RESULT 2026-10-07 16:46:52Z

```

upstream
/usr/bin/avconvert
/usr/bin/afinfo
/usr/bin/mdls
/usr/bin/osascript
/usr/bin/expect
/opt/homebrew/bin/brew
ModuleNotFoundError: No module named 'imageio_ffmpeg'
-rw-r--r--@ 1 aadi wheel 3548210 Oct 7 22:16 t.png
-rw-----@ 1 aadi staff 2782377 Oct 7 22:16 t.mov
kMDItemDurationSeconds = (null)
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4

```

TOOL CALL 2026-10-07 16:46:55Z

```

[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && ainfo
t.mov 2>&1 | head; avconvert --preset PresetMediumQuality --source t.mov --output t.mp4 --replace 2>&1 | tail -3; ls
-la t.mp4; mdls -name kMDItemDurationSeconds t.mp4; which swift

```

TOOL CALL 2026-10-07 16:46:55Z

```

[Read]
{
  "file_path":

```

```
"/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/t.png"
}
```

TOOL RESULT 2026-10-07 16:46:57Z

```
Fail: AudioFileOpenURL failed
File:          t.mov
avconvert completed with error:0.

-rw-r--r--@ 1 aadi  wheel  133540 Oct  7 22:16 t.mp4
kMDItemDurationSeconds = (null)
/usr/bin/swift
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL RESULT 2026-10-07 16:46:58Z

[image omitted]

USER 2026-10-07 16:46:58Z

[Image: original 3600x2338, displayed at 2000x1299. Multiply coordinates by 1.80 to map to original image.]

TOOL CALL 2026-10-07 16:47:04Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && rm -f
t.png t.mov t.mp4 && ls; python3 -c "import Quartz" 2>&1 | tail -1
```

TOOL RESULT 2026-10-07 16:47:04Z

```
upstream
ModuleNotFoundError: No module named 'Quartz'
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

CLAUDE 2026-10-07 16:47:07Z

Deleted the test captures. Next I'm setting up a dedicated Terminal window and testing a window-only capture, so nothing else on screen gets recorded.

TOOL CALL 2026-10-07 16:47:09Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && cat >
winid.swift <<'EOF'
import Cocoa
let list = CGWindowListCopyWindowInfo([.optionOnScreenOnly], kCGNullWindowID) as! [[String: Any]]
for w in list {
    if (w["kCGWindowOwnerName"] as? String) == "Terminal" {
        print(w["kCGWindowNumber"] as! Int, w["kCGWindowBounds"] ?? "")
    }
}
EOF
osascript -e 'tell application "Terminal"
    activate
    set w to do script "clear; echo window-capture-test"
    set bounds of front window to {100, 100, 900, 560}
end tell' ; sleep 2; swiftc -o winid winid.swift 2>&1 | tail -3; ./winid
```

TOOL RESULT 2026-10-07 16:47:22Z

```
7231 {
    Height = 460;
    Width = 800;
    X = 100;
    Y = 100;
}
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL CALL 2026-10-07 16:47:24Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad &&
screencapture -x -o -17231 win.png && ls -la win.png && screencapture -v -V 2 -17231 win.mov 2>&1; ls -la win.mov
```

TOOL CALL 2026-10-07 16:47:25Z

```
[Read]
{
  "file_path":
  "/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/win.png"
}
```

TOOL RESULT 2026-10-07 16:47:27Z

```
-rw-r--r--@ 1 aadi  wheel  169262 Oct  7 22:17 win.png
-rw-----@ 1 aadi  staff  35436 Oct  7 22:17 win.mov
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL RESULT 2026-10-07 16:47:27Z

[image omitted]

TOOL CALL 2026-10-07 16:47:32Z

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal" &&
U=/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream &&
mkdir -p Lab4/mastermind && cp $U/README.md $U/main.py $U/game.py $U/logic.py $U/requirements.txt Lab4/mastermind/ &&
diff -r $U Lab4/mastermind -x .git && echo identical; python3 - <<'EOF'
import random
```

```

for s in range(100000):
    random.seed(s)
    if [str(random.randint(1,6)) for _ in range(4)] == list("4413"):
        print("seed",s); break
EOF
cd Lab4/mastermind && python3 -c "
from logic import feedback
print(feedback(list('4413'), list('4444')))"

```

TOOL RESULT 2026-10-07 16:47:33Z

```

identical
seed 0
(2, 2)

```

CLAUDE 2026-10-07 16:47:35Z

The defect is reproduced: `feedback(4413, 4444)` returns `(2, 2)` where the correct answer is `(2, 0)`, and `random.seed(0)` produces code 4413. I'll commit the untouched baseline.

TOOL CALL 2026-10-07 16:47:36Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal" && git add Lab4 && git status --short && git commit -q -m "Lab 4: Add Mastermind baseline

```

Untouched copy of SETAPESU26/20_mastermind (63e3c15) as the starting point.

Co-Authored-By: Claude Sonnet 5.5 <noreply@anthropic.com>" && git log --oneline | head -3

TOOL RESULT 2026-10-07 16:47:37Z

```

A Lab4/mastermind/README.md
A Lab4/mastermind/game.py
A Lab4/mastermind/logic.py
A Lab4/mastermind/main.py
A Lab4/mastermind/requirements.txt
33abfde Lab 4: Add Mastermind baseline
46563d3 Add Lab 3: Component Modelling & Architectural Pattern Selection
edbf4b Add Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

```

TOOL CALL 2026-10-07 16:48:11Z

```

[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && cat >
vinfo.swift <<'EOF'
import AVFoundation
import AppKit
let a = CommandLine.arguments
let asset = AVURLAsset(url: URL(fileURLWithPath: a[1]))
let d = asset.duration.seconds
print(String(format: "duration: %.2f s", d))
for t in a.dropFirst(2) {
    let g = AVAssetImageGenerator(asset: asset)
    g.appliesPreferredTrackTransform = true
    g.requestedTimeToleranceBefore = .zero; g.requestedTimeToleranceAfter = .zero
    if let img = try? g.copyCGImage(at: CMTime(seconds: Double(t)!, preferredTimescale: 600), actualTime: nil) {
        let rep = NSBitmapImageRep(cgImage: img)
        let png = rep.representation(using: .png, properties: [:])!
        try! png.write(to: URL(fileURLWithPath: "frame_\(t).png"))
        print("frame_\(t).png \((img.width)x\((img.height))")
    }
}
EOF
swiftc -o vinfo vinfo.swift 2>&1 | grep -v warning | head; ./vinfo win.mov 1.0
cat > demo.py <<'EOF'
"""Recording helper (not part of the submission): runs the real game on a pty with scripted typing."""
import os, pty, select, sys, time, fcntl, termios, struct

```

CYAN, RED, GREEN, DIM, RST = "\033[36m", "\033[1;31m", "\033[1;32m", "\033[2m", "\033[0m"

```

def build(label):
    acts, t = [], 0.0
    def say(txt, at=None):
        nonlocal t
        if at is not None: t = at
        acts.append((t, "print", txt + "\r\n"))
    def type_(text, at=None, cps=9):
        nonlocal t
        if at is not None: t = at
        for ch in text:
            acts.append((t, "send", ch)); t += 1.0 / cps
            acts.append((t + 0.15, "send", "\r")); t += 0.15
        return acts, say, type_

def scenario(label):
    acts, say, type_ = build(label)
    if label == "before":
        seed, hdr = 0, [f"{RED}BEFORE{RST} - original, unmodified game (seed 0 -> secret 4413, shown for the demo
only)"]
        for h in hdr: say(h, 0.0)
        type_("4444", 1.6)
        say(f"{RED}^ BUG: secret 4413 vs 4444 should be Exact 2, Partial 0{RST}", 3.5)
        type_("1111", 4.6)
        say(f"{RED}^ BUG: secret 4413 vs 1111 should be Exact 1, Partial 0{RST}", 6.5)
        type_("q", 8.0)

```

```

    return seed, acts

def main():
    label, gamedir = sys.argv[1], sys.argv[2]
    seed, acts = scenario(label)
    acts.sort(key=lambda a: a[0])
    os.write(1, b"\033[2J\033[H")
    pid, fd = pty.fork()
    if pid == 0:
        os.chdir(gamedir)
        code = ("import random,runpy,sys; random.seed(%d); sys.argv=['main.py']; "
                "runpy.run_path('main.py', run_name='__main__')" % seed)
        os.execvp("python3", ["python3", "-c", code])
    fcntl.ioctl(fd, termios.TIOCSWINSZ, struct.pack("HHHH", 29, 100, 0, 0))
    start, i, alive = time.time(), 0, True
    while alive or i < len(acts):
        now = time.time() - start
        while i < len(acts) and acts[i][0] <= now:
            _, kind, data = acts[i]; i += 1
            if kind == "send": os.write(fd, data.encode())
            else: os.write(1, data.encode())
        wait = max(0.0, (acts[i][0] - now) if i < len(acts) else 0.05)
        if alive:
            r, _, _ = select.select([fd], [], [], min(wait, 0.05))
            if r:
                try:
                    out = os.read(fd, 4096)
                    if not out: alive = False
                    else: os.write(1, out)
                except OSError: alive = False
        else:
            time.sleep(min(wait, 0.05))
    os.waitpid(pid, 0)

main()
EOF
echo ok

```

TOOL RESULT 2026-10-07 16:48:14Z

```

3 | let a = CommandLine.arguments
4 | let asset = AVURLAsset(url: URL(fileURLWithPath: a[1]))
5 | let d = asset.duration.seconds
6 | print(String(format: "duration: %.2f s", d))
7 | for t in a.dropFirst(2) {

9 |     g.appliesPreferredTrackTransform = true
10 |     g.requestedTimeToleranceBefore = .zero; g.requestedTimeToleranceAfter = .zero
11 |     if let img = try? g.copyCGImage(at: CMTime(seconds: Double(t)!, preferredTimescale: 600), actualTime: nil) {
12 |         let rep = NSBitmapImageRep(cgImage: img)
duration: 2.00 s
frame_1.0.png 1692x1012
ok
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4

```

CLAUDE 2026-10-07 16:48:17Z

Helper tools work. I'll do a dry run of the "before" scenario in the Terminal window (not recording yet) and check what it looks like.

TOOL CALL 2026-10-07 16:48:19Z

```

[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && S=$PWD &&
osascript -e "tell application \"Terminal\" to do script \"PS1='\$ ' ; clear; python3 $S/demo.py before
'/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind\" in front window" >/dev/null; sleep 11; screencapture -x -o
-17231 win.png

```

TOOL CALL 2026-10-07 16:48:20Z

```

[Read]
{
  "file_path":
    "/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/win.png"
}

```

TOOL RESULT 2026-10-07 16:48:31Z

Shell cwd was reset to /Users/aadi/Desktop/SE lab 4

TOOL RESULT 2026-10-07 16:48:31Z

[image omitted]

CLAUDE 2026-10-07 16:48:40Z

The bug shows clearly, but my annotations interleave awkwardly with the game's prompt. I'll put the expected values in the header and the red summary after the game exits, then record for real.

TOOL CALL 2026-10-07 16:48:44Z

```

[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && python3 -
<<'EOF'
s = open("demo.py").read()
a = s.index('    if label == "before":')
b = s.index('        return seed, acts')
new = '''    if label == "before":

```

```

seed = 0
say(f"{RED}BEFORE{RST} - original, unmodified game (seed 0 -> secret 4413, shown for the demo only)", 0.0)
say(f"{CYAN}Correct feedback would be: 4444 -> Exact 2, Partial 0 | 1111 -> Exact 1, Partial 0{RST}",
0.1)

type_("4444", 2.0)
type_("1111", 4.2)
type_("q", 6.2)
say(f"{RED}BUG: partial counts are 2 and 3 - the same code digits are counted repeatedly{RST}", 7.0)
'''
open("demo.py", "w").write(s[:a]+new+s[b:])
EOF
S=$PWD; GD='/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind'
osascript -e 'tell application "Terminal" to do script "PS1='''$ '""'; clear" in front window' >/dev/null; sleep 1
screencapture -v -V 10 -17231 before_raw.mov &
sleep 1.2
osascript -e "tell application \"Terminal\" to do script \"python3 $S/demo.py before '$GD'\" in front window"
>/dev/null
wait; ls -la before_raw.mov; ./vinfo before_raw.mov 0.5 3.5 6.0 9.0

```

TOOL RESULT 2026-10-07 16:48:56Z

```

-rw-----@ 1 aadi staff 996653 Oct 7 22:18 before_raw.mov
duration: 10.02 s
frame_0.5.png 1692x1012
frame_3.5.png 1692x1012
frame_6.0.png 1692x1012
frame_9.0.png 1692x1012
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4

```

TOOL CALL 2026-10-07 16:48:58Z

```

[Read]
{
  "file_path":
"/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/frame_9.0.png"
}

```

TOOL RESULT 2026-10-07 16:48:58Z

[image omitted]

TOOL CALL 2026-10-07 16:48:59Z

```

[Read]
{
  "file_path":
"/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/frame_0.5.png"
}

```

TOOL RESULT 2026-10-07 16:48:59Z

[image omitted]

TOOL CALL 2026-10-07 16:49:11Z

```

[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && python3 -
<<'EOF'
s = open("demo.py").read()
s = s.replace('''' say(f"{RED}BEFORE{RST} - original, unmodified game (seed 0 -> secret 4413, shown for the
demo only)", 0.0)
say(f"{CYAN}Correct feedback would be: 4444 -> Exact 2, Partial 0 | 1111 -> Exact 1, Partial 0{RST}",
0.1)
type_("4444", 2.0)
type_("1111", 4.2)
type_("q", 6.2)
say(f"{RED}BUG: partial counts are 2 and 3 - the same code digits are counted repeatedly{RST}", 7.0)
'''',''' pre.append(f"{RED}BEFORE{RST} - original, unmodified game (seed 0 -> secret 4413, shown for the demo
only)")
pre.append(f"{CYAN}Correct feedback would be: 4444 -> Exact 2, Partial 0 | 1111 -> Exact 1, Partial
0{RST}")
type_("4444", 1.4)
type_("1111", 3.4)
type_("q", 5.4)
say(f"{RED}BUG: partial counts are 2 and 3 - the same code digits are counted repeatedly{RST}", 6.2)
'''')
s = s.replace(" acts, say, type_ = build(label)\n", " acts, say, type_ = build(label)\n pre = []\n")
s = s.replace(" return seed, acts", " return seed, acts, pre")
s = s.replace(" seed, acts = scenario(label)", " seed, acts, pre = scenario(label)")
s = s.replace('os.write(1, b"\033[2J\033[H"]', 'os.write(1, b"\033[2J\033[H"]\n for line in pre: os.write(1,
(line + "\r\n").encode())')
open("demo.py", "w").write(s)
EOF
grep -n "pre" demo.py | head
S=$PWD; GD='/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind'
osascript -e 'tell application "Terminal" to do script "PS1='''$ '""'; clear" in front window' >/dev/null; sleep 1
screencapture -v -V 10 -17231 before_raw.mov &
sleep 0.6
osascript -e "tell application \"Terminal\" to do script \"python3 $S/demo.py before '$GD'\" in front window"
>/dev/null
wait; ./vinfo before_raw.mov 1.0 4.5 9.5

```

TOOL RESULT 2026-10-07 16:49:23Z

```

22: pre = []
25: pre.append(f"{RED}BEFORE{RST} - original, unmodified game (seed 0 -> secret 4413, shown for the demo
only)")
26: pre.append(f"{CYAN}Correct feedback would be: 4444 -> Exact 2, Partial 0 | 1111 -> Exact 1, Partial

```

```
0{RST}")
31:     return seed, acts, pre
35:     seed, acts, pre = scenario(label)
38:     for line in pre: os.write(1, (line + "\r\n").encode())
screencapture: Failed to save to final location before_raw.mov
duration: 10.02 s
frame_1.0.png 1692x1012
frame_4.5.png 1692x1012
frame_9.5.png 1692x1012
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

CLAUDE 2026-10-07 16:49:25Z

The file already existed, so `screencapture` refused to overwrite it and I read the old take. Removing it and recording again.

TOOL CALL 2026-10-07 16:49:27Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && rm -f
before_raw.mov frame_*.png
S=$PWD; GD='/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind'
osascript -e 'tell application "Terminal" to do script "PS1='''$ ''''; clear" in front window' >/dev/null; sleep 1
screencapture -v -V 10 -l7231 before_raw.mov &
sleep 0.6
osascript -e "tell application \"Terminal\" to do script \"python3 $S/demo.py before '$GD'\" in front window"
>/dev/null
wait; ls -la before_raw.mov; ./vinfo before_raw.mov 1.0 4.5 9.5
```

TOOL RESULT 2026-10-07 16:49:40Z

```
-rw-----@ 1 aadi  staff  1374663 Oct  7 22:19 before_raw.mov
duration: 10.01 s
frame_1.0.png 1692x1012
frame_4.5.png 1692x1012
frame_9.5.png 1692x1012
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL CALL 2026-10-07 16:49:42Z

```
[Read]
{
  "file_path":
"/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/frame_4.5.png"
}
```

TOOL RESULT 2026-10-07 16:49:42Z

[image omitted]

TOOL CALL 2026-10-07 16:49:43Z

```
[Read]
{
  "file_path":
"/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/frame_9.5.png"
}
```

TOOL RESULT 2026-10-07 16:49:43Z

[image omitted]

TOOL CALL 2026-10-07 16:49:47Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && rm -f
before_10s.mp4 && avconvert --preset PresetHighestQuality --source before_raw.mov --output before_10s.mp4 2>&1 | tail
-1; ls -la before_10s.mp4; ./vinfo before_10s.mp4 9.5; file before_10s.mp4; python3 - <<'EOF'
import subprocess
print(subprocess.run(["mdls", "-name", "kMDItemCodecs", "before_10s.mp4"], capture_output=True, text=True).stdout)
EOF
```

TOOL RESULT 2026-10-07 16:49:50Z

```
-rw-r--r--@ 1 aadi  wheel  1374303 Oct  7 22:19 before_10s.mp4
duration: 10.01 s
frame_9.5.png 1692x1012
before_10s.mp4: ISO Media, MP4 v2 [ISO 14496-14]
kMDItemCodecs = (null)
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL CALL 2026-10-07 16:49:53Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && grep -c
-a "avc1" before_10s.mp4; mkdir -p keep && cp before_10s.mp4 keep/
```

TOOL RESULT 2026-10-07 16:49:53Z

```
1
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

CLAUDE 2026-10-07 16:50:07Z

The "before" video is saved (10.01 s, H.264). Now Task 1, starting with the fix in `logic.py`: a two-pass algorithm that resolves exact matches first, then pairs only the leftover symbols.

TOOL CALL 2026-10-07 16:50:09Z

```
[Write]
```

```
{
  "file_path": "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/logic.py",
  "content": "from collections import Counter\n\ndef feedback(code, guess):\n    \"\"\"Return (exact, partial) for a\n    guess against the secret code.\"\"\"\n    Each code position is counted at most once:\n    1. Exact matches (same\n    symbol, same position) are resolved first.\n    2. Positions that were not exact matches are then paired up by\n    symbol,\n    so a code occurrence already used by an exact match - or by an\n    earlier partial match - can\n    never contribute again.\n    exact = sum(c == g for c, g in zip(code, guess))\n    unmatched_code =\n    Counter(c for c, g in zip(code, guess) if c != g)\n    unmatched_guess = Counter(g for c, g in zip(code, guess) if c\n    != g)\n    partial = sum((unmatched_code & unmatched_guess).values())\n    return exact, partial\n"}
}
```

TOOL CALL 2026-10-07 16:50:17Z

```
[Write]
{
  "file_path": "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/tests/test_mastermind.py",
  "content": "\"\"\"Tests for the Mastermind game. Run from Lab4/mastermind with:\n\n    python3 -m unittest discover\n    -s tests -v\"\"\"\nimport itertools\nimport os\nimport sys\nimport unittest\nimport sys.path.insert(0,\n    os.path.join(os.path.dirname(__file__), '..'))\nfrom logic import feedback\n\ndef fb(code, guess):\n    return\n    feedback(list(code), list(guess))\n\nclass FeedbackTests(unittest.TestCase):\n    \"\"\"Task 1: duplicate-aware\n    feedback.\"\"\"\n\n    def test_original_defect_case(self):\n        # Originally reported (2, 2): both 4s in the code\n        were already used.\n        self.assertEqual(fb(\"4413\", \"4444\"), (2, 0))\n\n    def test_all_exact(self):\n        self.assertEqual(fb(\"1234\", \"1234\"), (4, 0))\n        self.assertEqual(fb(\"4413\", \"4413\"), (4, 0))\n\n    def test_no_matches(self):\n        self.assertEqual(fb(\"1111\", \"1111\"), (4, 0))\n\n    def test_all_partial(self):\n        self.assertEqual(fb(\"1234\", \"5656\"), (0, 0))\n\n    def test_mixed_exact_and_partial(self):\n        self.assertEqual(fb(\"1234\", \"1325\"), (1, 2))\n\n    def test_repeated_symbols_in_guess_only(self):\n        self.assertEqual(fb(\"1234\", \"1243\"), (2, 2))\n\n    def test_repeated_symbols_in_code_only(self):\n        self.assertEqual(fb(\"1234\", \"1115\"), (1, 0))\n\n    def test_repeated_symbols_in_both(self):\n        self.assertEqual(fb(\"1234\", \"2111\"), (0, 2))\n\n    def test_exact_resolved_before_partial(self):\n        # The\n        leading 1 in the guess must not steal the code's only 1,\n        # which is matched exactly by the guess's last\n        position.\n        self.assertEqual(fb(\"2221\", \"1111\"), (1, 0))\n        self.assertEqual(fb(\"1222\", \"2221\"),\n            (0, 4)[0] + (1, 2))\n\n    def test_longer_codes(self):\n        self.assertEqual(fb(\"123456\", \"654321\"), (0,\n            6))\n\n    def test_matches_independent_reference_exhaustively(self):\n        # Compare\n        against the textbook formula over every 4-symbol code/guess\n        from a 3-symbol alphabet (6,561 pairs, heavy on\n        repeated symbols).\n        for code in itertools.product(\"123\", repeat=4):\n            for guess in\n                itertools.product(\"123\", repeat=4):\n                exact = sum(c == g for c, g in zip(code, guess))\n                shared = sum(min(code.count(s), guess.count(s)) for s in \"123\")\n                self.assertEqual(feedback(list(code), list(guess)),\n                    (exact, shared - exact))\n\nif __name__ == \"__main__\":\n    unittest.main()\n"}
}
```

TOOL RESULT 2026-10-07 16:50:17Z

The file /Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/logic.py has been updated successfully. (file state is current in your context - no need to Read it back)

TOOL RESULT 2026-10-07 16:50:17Z

File created successfully at: /Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/tests/test_mastermind.py (file state is current in your context - no need to Read it back)

CLAUDE 2026-10-07 16:50:22Z

I left a garbled placeholder assertion in the test file; fixing it to the correct hand-computed value (1222 vs 2221 → exact 2, partial 2).

TOOL CALL 2026-10-07 16:50:23Z

```
[Edit]
{
  "replace_all": false,
  "file_path": "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/tests/test_mastermind.py",
  "old_string": "        self.assertEqual(fb(\"1222\", \"2221\"), (0, 4)[0] + (1, 2))",
  "new_string": "        self.assertEqual(fb(\"1222\", \"2221\"), (2, 2))"
}
```

TOOL RESULT 2026-10-07 16:50:23Z

The file /Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/tests/test_mastermind.py has been updated successfully. (file state is current in your context - no need to Read it back)

TOOL CALL 2026-10-07 16:50:25Z

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && python3 -m unittest discover -s tests -v 2>&1 | tail -20; echo ---; git stash -q -- logic.py && python3 -m unittest discover -s tests 2>&1 | tail -4; git stash pop -q && git status --short
```

TOOL RESULT 2026-10-07 16:50:26Z

```
test_all_exact (test_mastermind.FeedbackTests.test_all_exact) ... ok
test_all_partial (test_mastermind.FeedbackTests.test_all_partial) ... ok
test_exact_resolved_before_partial (test_mastermind.FeedbackTests.test_exact_resolved_before_partial) ... ok
test_longer_codes (test_mastermind.FeedbackTests.test_longer_codes) ... ok
test_matches_independent_reference_exhaustively
(test_mastermind.FeedbackTests.test_matches_independent_reference_exhaustively)
Compare against the textbook formula over every 4-symbol code/guess ... ok
test_mixed_exact_and_partial (test_mastermind.FeedbackTests.test_mixed_exact_and_partial) ... ok
test_no_matches (test_mastermind.FeedbackTests.test_no_matches) ... ok
test_original_defect_case (test_mastermind.FeedbackTests.test_original_defect_case) ... ok
```

```
test_repeated_symbols_in_both (test_mastermind.FeedbackTests.test_repeated_symbols_in_both) ... ok
test_repeated_symbols_in_code_only (test_mastermind.FeedbackTests.test_repeated_symbols_in_code_only) ... ok
test_repeated_symbols_in_guess_only (test_mastermind.FeedbackTests.test_repeated_symbols_in_guess_only) ... ok
```

Ran 11 tests in 0.055s

OK

Ran 11 tests in 0.608s

FAILED (failures=3628)
M logic.py
?? __pycache__/
?? tests/

CLAUDE 2026-10-07 16:50:30Z

The tests pass with the fix, and 3,628 subtests fail against the original `logic.py`, so they do catch the defect. Next a manual check, then I'll review the diff and commit.

TOOL CALL 2026-10-07 16:50:33Z

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && rm -rf __pycache__ tests/__pycache__ && export
PYTHONDONTWRITEBYTECODE=1 && python3 -c "
from logic import feedback
for c,g in [('4413','4444'),('4413','1111'),('1122','1212'),('1234','4321')]:
    print(c,g,feedback(list(c),list(g)))" && git diff --stat && git add logic.py tests/test_mastermind.py && git
status --short && git commit -q -m "Lab 4: Fix duplicate-aware feedback
```

Resolve exact matches first, then pair only the unmatched positions by
symbol, so each code position is counted at most once. Previously
feedback(4413, 4444) reported Partial 2 instead of 0.

Adds unittest coverage: all exact, no matches, mixed, repeated symbols
(guess-only, code-only, both), longer codes, and an exhaustive comparison
against an independent reference formula.

Co-Authored-By: Claude Sonnet 5.5 <noreply@anthropic.com>" && git log --oneline | head -3

TOOL RESULT 2026-10-07 16:50:33Z

```
4413 4444 (2, 0)
4413 1111 (1, 0)
1122 1212 (2, 2)
1234 4321 (0, 4)
Lab4/mastermind/logic.py | 17 ++++++-----
1 file changed, 14 insertions(+), 3 deletions(-)
M logic.py
A tests/test_mastermind.py
3b1a33a Lab 4: Fix duplicate-aware feedback
33abfde Lab 4: Add Mastermind baseline
46563d3 Add Lab 3: Component Modelling & Architectural Pattern Selection
```

TOOL CALL 2026-10-07 16:50:52Z

```
[Write]
{
  "file_path": "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/game.py",
  "content": "import random\nfrom logic import feedback\n\nPLAYING, WON, LOST, QUIT = \"playing\", \"won\", \"lost\", \"quit\"\n\n\nclass GameOver(Exception):\n    \"\"\"Raised when a guess is submitted after the game has already ended.\"\"\"\n\n\nclass Mastermind:\n    def __init__(self, code=None, max_turns=10):\n        self.code = code or str(random.randint(1, 6)) for _ in range(4)]\n        self.history = []\n        self.turns = max_turns\n        self.state = PLAYING\n        @property\n        def finished(self):\n            return self.state != PLAYING\n        def submit(self, guess):\n            \"\"\"Score one accepted guess and advance the game; return (exact, partial).\"\"\"\n            Raises GameOver, leaving all state untouched, once the game has ended.\n            if self.finished:\n                raise GameOver(f\"The game is already over ({self.state}).\")\n            exact, partial = feedback(self.code, guess)\n            self.history.append((\"\"\".join(guess), exact, partial))\n            self.turns -= 1\n            if exact == len(self.code):\n                self.state = WON\n            elif self.turns == 0:\n                self.state = LOST\n            return exact, partial\n        def quit(self):\n            if not self.finished:\n                self.state = QUIT\n        def run(self, read=input, write=print):\n            write(\"Mastermind - enter four digits from 1 to 6.\")\n            while not self.finished:\n                raw = read(f\"{self.turns} turns left > \").strip()\n                if raw.lower() == \"q\":\n                    self.quit()\n                    break\n                if len(raw) != 4 or any(ch not in \"123456\" for ch in raw):\n                    write(\"Enter exactly four digits from 1 to 6.\")\n                    continue\n                exact, partial = self.submit(list(raw))\n                write(\"Exact:\", exact, \"Partial:\", partial)\n                self.announce_result(write)\n        def announce_result(self, write=print):\n            code = \"\"\".join(self.code)\n            guesses = len(self.history)\n            if self.state == WON:\n                write(f\"Cracked the code in {guesses} guess{'es' if guesses != 1 else ''}!\")\n            elif self.state == LOST:\n                write(f\"Out of turns - you lose. The code was {code}\")\n            elif self.state == QUIT:\n                write(f\"You quit. The code was {code}\")\n            }
```

TOOL RESULT 2026-10-07 16:50:52Z

The file /Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/game.py has been updated successfully. (file state is current in your context - no need to Read it back)

TOOL CALL 2026-10-07 16:50:55Z

```
[Edit]
{
  "replace_all": false,
  "file_path": "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/game.py",
  "old_string": "        write(\"Exact:\", exact, \"Partial:\", partial)",
  "new_string": "        write(f\"Exact: {exact} Partial: {partial}\")"
```

```
}
```

TOOL RESULT 2026-10-07 16:50:55Z

The file /Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/game.py has been updated successfully. (file state is current in your context – no need to Read it back)

CLAUDE 2026-10-07 16:50:56Z

Now the lifecycle tests: immediate, multi-guess and final-turn wins, a loss, quitting, and proof that state and history don't change after the game ends.

TOOL CALL 2026-10-07 16:51:06Z

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && python3 - <<'EOF'
p = "tests/test_mastermind.py"
s = open(p).read()
s = s.replace("from logic import feedback\n", "from game import GameOver, LOST, Mastermind, PLAYING, QUIT, WON\nfrom
logic import feedback\n")
s = s.replace('

if __name__ == "__main__":', '''

def play(game, inputs):
    """Drive game.run() with scripted input; return (printed lines, unread inputs)."""
    queue, out = list(inputs), []

    def read(prompt):
        out.append(prompt)
        return queue.pop(0)

    game.run(read=read, write=out.append)
    return out, queue

class LifecycleTests(unittest.TestCase):
    """Task 2: win / loss / quit handling, history, guess limit, frozen state."""

    def make(self, max_turns=10):
        return Mastermind(code=list("1234"), max_turns=max_turns)

    def snapshot(self, game):
        return (game.state, game.turns, list(game.history))

    def test_new_game_is_playing(self):
        game = self.make()
        self.assertEqual((game.state, game.turns, game.history), (PLAYING, 10, []))
        self.assertFalse(game.finished)

    def test_immediate_win(self):
        game = self.make()
        out, _ = play(game, ["1234"])
        self.assertEqual(game.state, WON)
        self.assertEqual(game.turns, 9)
        self.assertEqual(game.history, [("1234", 4, 0)])
        self.assertIn("Cracked the code in 1 guess!", out)

    def test_win_after_several_guesses(self):
        game = self.make()
        out, _ = play(game, ["1111", "2143", "1243", "1234"])
        self.assertEqual(game.state, WON)
        self.assertEqual(game.turns, 6)
        self.assertEqual(game.history, [("1111", 1, 0), ("2143", 0, 4),
                                         ("1243", 2, 2), ("1234", 4, 0)])
        self.assertIn("Cracked the code in 4 guesses!", out)

    def test_win_on_final_turn_is_a_win_not_a_loss(self):
        game = self.make(max_turns=3)
        play(game, ["1111", "2222", "1234"])
        self.assertEqual(game.state, WON)
        self.assertEqual(game.turns, 0)
        self.assertEqual(len(game.history), 3)

    def test_loss_after_all_guesses_used(self):
        game = self.make(max_turns=3)
        out, _ = play(game, ["1111", "2222", "3333"])
        self.assertEqual(game.state, LOST)
        self.assertEqual(game.turns, 0)
        self.assertEqual(len(game.history), 3)
        self.assertIn("Out of turns - you lose. The code was 1234", out)

    def test_quit_ends_game_and_keeps_history(self):
        game = self.make()
        out, _ = play(game, ["1111", "q"])
        self.assertEqual(game.state, QUIT)
        self.assertEqual(game.turns, 9)
        self.assertEqual(game.history, [("1111", 1, 0)])
        self.assertIn("You quit. The code was 1234", out)

    def test_quit_is_case_insensitive_and_costs_no_turn(self):
        game = self.make()
        play(game, ["Q"])
        self.assertEqual((game.state, game.turns, game.history), (QUIT, 10, []))

    def test_guess_limit_is_enforced(self):
```

```

        game = self.make(max_turns=2)
        game.submit(list("1111"))
        game.submit(list("2222"))
        self.assertEqual(game.turns, 0)
        with self.assertRaises(GameOver):
            game.submit(list("3333"))
        self.assertEqual(len(game.history), 2)

    def test_state_frozen_after_win_loss_and_quit(self):
        for max_turns, script in [(10, ["1234"]), (2, ["1111", "2222"]), (10, ["1111", "q"])]:
            with self.subTest(script=script):
                game = self.make(max_turns=max_turns)
                play(game, script)
                before = self.snapshot(game)
                for extra in ("1234", "9999", "1111"):
                    with self.assertRaises(GameOver):
                        game.submit(list(extra))
                game.quit()
                self.assertEqual(self.snapshot(game), before)

    def test_run_reads_no_input_after_game_ends(self):
        game = self.make()
        out, unread = play(game, ["1234", "1111", "1111"])
        self.assertEqual(unread, ["1111", "1111"])
        self.assertEqual(len(game.history), 1)

    def test_history_is_intact_when_game_is_lost(self):
        game = self.make(max_turns=3)
        play(game, ["1111", "4321", "1243"])
        self.assertEqual(game.history, [("1111", 1, 0), ("4321", 0, 4), ("1243", 2, 2)])

if __name__ == "__main__":
    open(p, "w").write(s)
    EOF
    python3 -m unittest discover -s tests 2>&1 | tail -6; printf '4413 etc: ' ; printf '4444\n1111\nq\n' | python3 -c "
import random; random.seed(0)
from game import Mastermind; Mastermind().run()"

```

TOOL RESULT 2026-10-07 16:51:06Z

```

.....
-----
Ran 22 tests in 0.054s

OK
4413 etc: Mastermind - enter four digits from 1 to 6.
10 turns left > Exact: 2 Partial: 0
9 turns left > Exact: 1 Partial: 0
8 turns left > You quit. The code was 4413

```

CLAUDE 2026-10-07 16:51:09Z

That notice is just my own edit. All 22 tests pass, and the manual run shows the fixed feedback (Exact 2 / Partial 0) and the new quit message. Reviewing the diff and committing Task 2.

TOOL CALL 2026-10-07 16:51:11Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && rm -rf __pycache__ tests/__pycache__ && git status
--short && git diff --stat && git add game.py tests/test_mastermind.py && git commit -q -m "Lab 4: Complete game
lifecycle

```

Add an explicit game state (playing/won/lost/quit) and a submit() method that scores one accepted guess, enforces the guess limit, and raises GameOver without touching state once the game has ended. run() now stops reading input when the game finishes and announces a win, loss or quit. Guess history is kept intact across all outcomes.

Tests cover immediate, multi-guess and final-turn wins, loss, quitting, and unchanged state/history after termination.

Co-Authored-By: Claude Sonnet 5.5 <noreply@anthropic.com>" && git log --oneline | head -2

TOOL RESULT 2026-10-07 16:51:11Z

```

M game.py
M tests/test_mastermind.py
Lab4/mastermind/game.py | 72 ++++++-----
Lab4/mastermind/tests/test_mastermind.py | 105 ++++++-----
2 files changed, 159 insertions(+), 18 deletions(-)
a770f86 Lab 4: Complete game lifecycle
3b1a33a Lab 4: Fix duplicate-aware feedback

```

TOOL CALL 2026-10-07 16:51:39Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && python3 - <<'EOF'
s = open("game.py").read()

s = s.replace('import random
from logic import feedback

PLAYING, WON, LOST, QUIT = "playing", "won", "lost", "quit"
'', 'import random
from dataclasses import dataclass

```

```

from logic import feedback

PLAYING, WON, LOST, QUIT = "playing", "won", "lost", "quit"

@dataclass(frozen=True)
class Difficulty:
    name: str
    length: int      # positions in the secret code
    symbols: int      # code symbols are the digits 1..symbols
    max_turns: int    # guesses allowed

    @property
    def alphabet(self):
        return "".join(str(n) for n in range(1, self.symbols + 1))

    def describe(self):
        return f"{self.length} digits from 1-{self.symbols}, {self.max_turns} guesses"

EASY = Difficulty("easy", 3, 4, 12)
MEDIUM = Difficulty("medium", 4, 6, 10)    # the original game
HARD = Difficulty("hard", 5, 8, 8)
DIFFICULTIES = (EASY, MEDIUM, HARD)

def choose_difficulty(read=input, write=print):
    """Ask for a difficulty by number or name; return None if the player quits."""
    write("Choose a difficulty:")
    for number, level in enumerate(DIFFICULTIES, 1):
        write(f" {number}) {level.name:<6} - {level.describe()}")
    while True:
        choice = read("Difficulty (number or name, q to quit) > ").strip().lower()
        if choice == "q":
            return None
        for number, level in enumerate(DIFFICULTIES, 1):
            if choice in (str(number), level.name):
                return level
        write(f"Please choose 1-{len(DIFFICULTIES)} or a difficulty name.")
'''

s = s.replace(''''    def __init__(self, code=None, max_turns=10):
    self.code = code or [str(random.randint(1, 6)) for _ in range(4)]
    self.history = []
    self.turns = max_turns
    self.state = PLAYING
''', '''    def __init__(self, difficulty=MEDIUM, code=None):
    self.difficulty = difficulty
    self.code = code or [str(random.randint(1, difficulty.symbols))
        for _ in range(difficulty.length)]
    self.history = []
    self.turns = difficulty.max_turns
    self.state = PLAYING
''')

s = s.replace(''''        write("Mastermind - enter four digits from 1 to 6.")
''', '''        level = self.difficulty
        write(f"Mastermind ({level.name}) - enter {level.length} digits from 1 to {level.symbols}.")
''')

s = s.replace(''''        if len(row) != 4 or any(ch not in "123456" for ch in row):
        write("Enter exactly four digits from 1 to 6.")
        continue''', '''        if len(row) != level.length or any(ch not in level.alphabet for ch in
row):
        write(f"Enter exactly {level.length} digits from 1 to {level.symbols}.")
        continue''')

open("game.py", "w").write(s)

open("main.py", "w").write(''''from game import Mastermind, choose_difficulty

if __name__ == "__main__":
    level = choose_difficulty()
    if level:
        Mastermind(level).run()
''')
EOF
git diff main.py; sed -n 60,100p game.py

```

TOOL RESULT 2026-10-07 16:51:39Z

```

diff --git a/Lab4/mastermind/main.py b/Lab4/mastermind/main.py
index 174d300..61ca2f4 100644
--- a/Lab4/mastermind/main.py
+++ b/Lab4/mastermind/main.py
@@ -1,4 +1,6 @@
-from game import Mastermind
+from game import Mastermind, choose_difficulty

 if __name__ == "__main__":
-    Mastermind().run()
+    level = choose_difficulty()
+    if level:
+        Mastermind(level).run()

 def submit(self, guess):
     """Score one accepted guess and advance the game; return (exact, partial).

```

```

    Raises GameOver, leaving all state untouched, once the game has ended.
    """
    if self.finished:
        raise GameOver(f"The game is already over ({self.state}).")
    exact, partial = feedback(self.code, guess)
    self.history.append("".join(guess), exact, partial)
    self.turns -= 1
    if exact == len(self.code):
        self.state = WON
    elif self.turns == 0:
        self.state = LOST
    return exact, partial

def quit(self):
    if not self.finished:
        self.state = QUIT

def run(self, read=input, write=print):
    level = self.difficulty
    write(f"Mastermind ({level.name}) - enter {level.length} digits from 1 to {level.symbols}.")
    while not self.finished:
        raw = read(f"{self.turns} turns left > ").strip()
        if raw.lower() == "q":
            self.quit()
            break
        if len(raw) != level.length or any(ch not in level.alphabet for ch in raw):
            write(f"Enter exactly {level.length} digits from 1 to {level.symbols}.")
            continue
        exact, partial = self.submit(list(raw))
        write(f"Exact: {exact} Partial: {partial}")
    self.announce_result(write)

def announce_result(self, write=print):
    code = "".join(self.code)
    guesses = len(self.history)
    if self.state == WON:
        write(f"Cracked the code in {guesses} guess{'es' if guesses != 1 else ''}!")

```

TOOL CALL 2026-10-07 16:51:54Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && python3 - <<'EOF'
p = "tests/test_mastermind.py"
s = open(p).read()
s = s.replace("from game import GameOver, LOST, Mastermind, PLAYING, QUIT, WON\n",
    "import random\n\nfrom game import (DIFFICULTIES, EASY, HARD, LOST, MEDIUM, PLAYING, QUIT, WON,\n
Difficulty, GameOver, Mastermind, choose_difficulty)\n")
s = s.replace("import itertools\nimport os\nimport sys\nimport unittest\n", "import itertools\nimport os\nimport
random\nimport sys\nimport unittest\n")
s = s.replace("import random\n\nfrom game import", "from game import")
s = s.replace(''''        return Mastermind(code=list("1234"), max_turns=max_turns)''',
'''        level = Difficulty("test", 4, 6, max_turns)
        return Mastermind(level, code=list("1234"))''')
s = s.replace(''''

if __name__ == "__main__":'''

def run_menu(inputs):
    queue, out = list(inputs), []
    level = choose_difficulty(read=lambda prompt: queue.pop(0), write=out.append)
    return level, out

class DifficultyTests(unittest.TestCase):
    """Task 3: difficulty modes vary length, symbol range and guess limit."""

    def test_modes_differ_in_every_setting(self):
        for attr in ("length", "symbols", "max_turns"):
            values = [getattr(level, attr) for level in DIFFICULTIES]
            self.assertEqual(len(set(values)), len(values), attr)

    def test_medium_is_the_original_game(self):
        self.assertEqual((MEDIUM.length, MEDIUM.symbols, MEDIUM.max_turns), (4, 6, 10))

    def test_default_game_is_medium(self):
        self.assertEqual(Mastermind().difficulty, MEDIUM)

    def test_generated_codes_are_valid_for_every_mode(self):
        for level in DIFFICULTIES:
            for seed in range(200):
                random.seed(seed)
                game = Mastermind(level)
                with self.subTest(level=level.name, seed=seed):
                    self.assertEqual(len(game.code), level.length)
                    self.assertTrue(all(c in level.alphabet for c in game.code))
                    self.assertEqual(game.turns, level.max_turns)

    def test_codes_can_contain_repeated_symbols(self):
        for level in DIFFICULTIES:
            repeated = False
            for seed in range(200):
                random.seed(seed)
                code = Mastermind(level).code
                repeated = repeated or len(set(code)) < len(code)
            self.assertTrue(repeated, level.name)

```

```

def test_seed_zero_still_gives_the_original_code(self):
    random.seed(0)
    self.assertEqual("".join(Mastermind(MEDIUM).code), "4413")

def test_guess_validation_follows_the_mode(self):
    cases = [
        (EASY, "123", True), (EASY, "1234", False), (EASY, "12", False), (EASY, "125", False),
        (MEDIUM, "1236", True), (MEDIUM, "123", False), (MEDIUM, "12345", False), (MEDIUM, "1237", False),
        (HARD, "12348", True), (HARD, "1234", False), (HARD, "123456", False), (HARD, "12349", False),
    ]
    for level, guess, valid in cases:
        with self.subTest(level=level.name, guess=guess):
            game = Mastermind(level, code=list("1" * level.length))
            play(game, [guess, "q"])
            self.assertEqual(len(game.history), 1 if valid else 0)

def test_guess_limit_matches_the_mode(self):
    for level in DIFFICULTIES:
        with self.subTest(level=level.name):
            wrong = "2" * level.length
            game = Mastermind(level, code=list("1" * level.length))
            for _ in range(level.max_turns - 1):
                game.submit(list(wrong))
            self.assertEqual(game.state, PLAYING)
            game.submit(list(wrong))
            self.assertEqual((game.state, game.turns), (LOST, 0))
            self.assertEqual(len(game.history), level.max_turns)

def test_win_and_loss_in_every_mode(self):
    for level in DIFFICULTIES:
        code = list(level.alphabet[: level.length])
        wrong = "".join(reversed(code)) if level.length > 1 else "9"
        with self.subTest(level=level.name, outcome="win-on-last-turn"):
            game = Mastermind(level, code=code)
            play(game, [wrong] * (level.max_turns - 1) + ["".join(code)])
            self.assertEqual((game.state, game.turns), (WON, 0))
        with self.subTest(level=level.name, outcome="loss"):
            game = Mastermind(level, code=code)
            play(game, [wrong] * level.max_turns)
            self.assertEqual((game.state, game.turns), (LOST, 0))

def test_duplicate_aware_feedback_in_every_mode(self):
    cases = [
        (EASY, "113", "111", (2, 0)),
        (EASY, "121", "211", (1, 2)),
        (MEDIUM, "4413", "4444", (2, 0)),
        (MEDIUM, "1122", "2211", (0, 4)),
        (HARD, "11223", "12121", (1, 3)),
        (HARD, "88123", "18812", (0, 5)),
        (HARD, "88123", "88888", (2, 0)),
    ]
    for level, code, guess, expected in cases:
        with self.subTest(level=level.name, code=code, guess=guess):
            game = Mastermind(level, code=list(code))
            self.assertEqual(game.submit(list(guess)), expected)

def test_menu_accepts_number_or_name(self):
    for choice, expected in [("1", EASY), ("2", MEDIUM), ("3", HARD),
                             ("easy", EASY), (" Medium ", MEDIUM), ("HARD", HARD)]:
        with self.subTest(choice=choice):
            self.assertIs(run_menu([choice])[0], expected)

def test_menu_reprompts_on_bad_choice_and_can_quit(self):
    level, out = run_menu(["", "9", "extreme", "3"])
    self.assertIs(level, HARD)
    self.assertEqual(sum("Please choose" in line for line in out), 3)
    self.assertIsNone(run_menu(["q"])[0])

def test_menu_lists_every_mode(self):
    out = run_menu(["q"])[1]
    for level in DIFFICULTIES:
        self.assertTrue(any(level.name in line and level.describe() in line for line in out))

if __name__ == "__main__":
    open(p, "w").write(s)
    EOF
    python3 -m unittest discover -s tests 2>&1 | tail -30

```

TOOL RESULT 2026-10-07 16:51:55Z

```

~~~~~
AssertionError: Tuples differ: (2, 2) != (1, 3)

First differing element 0:
2
1

```

```

- (2, 2)
+ (1, 3)

```

```

=====
FAIL: test_duplicate_aware_feedback_in_every_mode
(test_mastermind.DifficultyTests.test_duplicate_aware_feedback_in_every_mode) (level='hard', code='88123',
guess='18812')

```



```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && python3 - <<'EOF'
# logic.py: add validation
s = open("logic.py").read()
s = s.replace('''from collections import Counter

'', '''from collections import Counter

class InvalidGuess(ValueError):
    """A guess that is malformed; its message is suitable to show the player."""

def validate_guess(raw, length, alphabet):
    """Return the guess as a list of symbols, or raise InvalidGuess.

    A guess must have exactly `length` characters, all drawn from `alphabet`.
    Membership is checked against the alphabet itself (not str.isdigit), so
    look-alike characters such as full-width or Arabic-Indic digits are rejected.
    """
    if not raw:
        raise InvalidGuess("Enter a guess, or q to quit.")
    if len(raw) != length:
        raise InvalidGuess(f"A guess needs exactly {length} digits - you entered {len(raw)}.")
    bad = sorted({ch for ch in raw if ch not in alphabet})
    if bad:
        shown = " ".join(repr(ch) for ch in bad)
        raise InvalidGuess(f"Only digits {alphabet[0]} to {alphabet[-1]} are allowed - not {shown}.")
    return list(raw)

'',1)
open("logic.py","w").write(s)

g = open("game.py").read()
g = g.replace("from logic import feedback\n", "from logic import InvalidGuess, feedback, validate_guess\n")
g = g.replace('''
    while True:
        choice = read("Difficulty (number or name, q to quit) > ").strip().lower()
        if choice == "q":'''', '''
    while True:
        try:
            choice = read("Difficulty (number or name, q to quit) > ").strip().lower()
        except (EOFError, KeyboardInterrupt):
            write("")
            return None
        if choice in QUIT_WORDS:'''')
g = g.replace(''''PLAYING, WON, LOST, QUIT = "playing", "won", "lost", "quit"
'', '''PLAYING, WON, LOST, QUIT = "playing", "won", "lost", "quit"
QUIT_WORDS = ("q", "quit")
''')
g = g.replace(''''
    def submit(self, guess):
        """Score one accepted guess and advance the game; return (exact, partial).

        Raises GameOver, leaving all state untouched, once the game has ended.
        """
        if self.finished:
            raise GameOver(f"The game is already over ({self.state}).")
        exact, partial = feedback(self.code, guess)
        self.history.append("".join(guess), exact, partial)'''', '''
    def submit(self, guess):
        """Score one accepted guess and advance the game; return (exact, partial).

        `guess` is a string (or sequence) of symbols. Raises GameOver once the
        game has ended and InvalidGuess for a malformed guess; in both cases all
        state is left untouched, so no turn is spent and no feedback is produced.
        """
        if self.finished:
            raise GameOver(f"The game is already over ({self.state}).")
        symbols = validate_guess("".join(guess), self.difficulty.length, self.difficulty.alphabet)
        exact, partial = feedback(self.code, symbols)
        self.history.append("".join(symbols), exact, partial)'''')
a = g.index("    def run(")
b = g.index("    def announce_result")
g = g[a:] + '''
    def history_lines(self):
        """The guess history as a table, one row per accepted guess."""
        width = max(5, 2 * self.difficulty.length - 1)
        lines = [f"{'#':>3} {'Guess':<{width}} {'Exact':>5} {'Partial':>7}"]
        for number, (guess, exact, partial) in enumerate(self.history, 1):
            lines.append(f"{'number':>3} {' '.join(guess):<{width}} {'exact':>5} {'partial':>7}")
        return lines

def run(self, read=input, write=print):
    level = self.difficulty
    write(f"Mastermind ({level.name}) - enter {level.length} digits "
          f"from 1 to {level.symbols}, or q to quit.")
    while not self.finished:
        try:
            raw = read(f"{self.turns} turns left > ").strip()
        except (EOFError, KeyboardInterrupt):
            write("")
            raw = "q"
        if raw.lower() in QUIT_WORDS:
            self.quit()
            break
        try:
            self.submit(raw)
        except InvalidGuess as problem:
```

```

        write(str(problem))
        continue
    for line in self.history_lines():
        write(line)
    self.announce_result(write)

''' + g[b:]
open("game.py","w").write(g)
EOF
python3 -m unittest discover -s tests 2>&1 | tail -5
printf 'easy\n111\n12\n1a3\n\n???n444\n11\n112 \n111\nquit\n' | python3 -c "
import random; random.seed(0)
import runpy, sys; sys.argv=['main.py']; runpy.run_path('main.py', run_name='__main__')"
```

TOOL RESULT 2026-10-07 16:52:48Z

```

.....
-----
Ran 35 tests in 0.069s

OK
Choose a difficulty:
  1) easy   - 3 digits from 1-4, 12 guesses
  2) medium - 4 digits from 1-6, 10 guesses
  3) hard   - 5 digits from 1-8, 8 guesses
Difficulty (number or name, q to quit) > Mastermind (easy) - enter 3 digits from 1 to 4, or q to quit.
12 turns left > # Guess Exact Partial
 1 1 1 1      1      0
11 turns left > A guess needs exactly 3 digits - you entered 2.
11 turns left > Only digits 1 to 4 are allowed - not 'a'.
11 turns left > Enter a guess, or q to quit.
11 turns left > Only digits 1 to 4 are allowed - not '?' '?' '?'.
11 turns left > # Guess Exact Partial
 1 1 1 1      1      0
 2 4 4 4      2      0
10 turns left > A guess needs exactly 3 digits - you entered 2.
10 turns left > # Guess Exact Partial
 1 1 1 1      1      0
 2 4 4 4      2      0
 3 1 1 2      0      1
 9 turns left > # Guess Exact Partial
 1 1 1 1      1      0
 2 4 4 4      2      0
 3 1 1 2      0      1
 4 1 1 1      1      0
 8 turns left > You quit. The code was 441
```

CLAUDE 2026-10-07 16:52:55Z

It works: malformed inputs are rejected with specific messages and don't touch the turn count, and the history table grows only with accepted guesses. Now the Task 4 tests.

TOOL CALL 2026-10-07 16:53:11Z

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && python3 - <<'EOF'
p = "tests/test_mastermind.py"
s = open(p).read()
s = s.replace("from logic import feedback\n", "from logic import InvalidGuess, feedback, validate_guess\n")
s = s.replace('

if __name__ == "__main__":', '

MALFORMED = [
    ("", "empty"), (" ", "blank"), ("123", "too short"), ("12345", "too long"),
    ("12a4", "letter"), ("abcd", "all letters"), ("1204", "zero"), ("1274", "above range"),
    ("12 3", "inner space"), ("1,23", "comma"), ("~123", "sign"), ("1.23", "decimal"),
    ("12\t3", "tab"), ("????", "full-width digits"), ("\u2013", "Arabic-Indic digits"),
    ("1234\n", "newline"),
]

class ValidationTests(unittest.TestCase):
    """Task 4: malformed guesses are rejected and never touch the game state."""

    def make(self, max_turns=10):
        return Mastermind(Difficulty("test", 4, 6, max_turns), code=list("1234"))

    def snapshot(self, game):
        return (game.state, game.turns, list(game.history))

    def test_validate_guess_accepts_well_formed_guesses(self):
        self.assertEqual(validate_guess("1234", 4, "123456"), list("1234"))
        self.assertEqual(validate_guess("6666", 4, "123456"), list("6666"))
        self.assertEqual(validate_guess("88888", 5, "12345678"), list("88888"))

    def test_validate_guess_rejects_malformed_guesses(self):
        for raw, why in MALFORMED:
            with self.subTest(why):
                with self.assertRaises(InvalidGuess):
                    validate_guess(raw, 4, "123456")

    def test_error_messages_say_what_is_wrong(self):
        for raw, expected in [("", "Enter a guess"), ("123", "exactly 4 digits - you entered 3"),
                              ("12a4", "'a'"), ("1274", "'7'")]:
```

```

        with self.subTest(row=row):
            with self.assertRaises(InvalidGuess) as caught:
                validate_guess(row, 4, "123456")
            self.assertIn(expected, str(caught.exception))

def test_submit_rejects_malformed_guess_without_changing_state(self):
    for raw, why in MALFORMED:
        with self.subTest(why):
            game = self.make()
            game.submit("1111")
            before = self.snapshot(game)
            with self.assertRaises(InvalidGuess):
                game.submit(raw)
            self.assertEqual(self.snapshot(game), before)

def test_invalid_input_in_run_spends_no_turn_and_prints_no_feedback(self):
    game = self.make()
    out, unread = play(game, [raw for raw, _ in MALFORMED] + ["q"])
    self.assertEqual(unread, [])
    self.assertEqual((game.state, game.turns, game.history), (QUIT, 10, []))
    self.assertFalse([line for line in out if "Exact" in line])
    self.assertEqual(sum(line.startswith("A guess needs") or line.startswith("Only digits")
                        or line.startswith("Enter a guess") for line in out), len(MALFORMED))

def test_prompt_turn_counter_does_not_move_on_invalid_input(self):
    game = self.make()
    out, _ = play(game, ["12", "abcd", "", "q"])
    prompts = [line for line in out if line.endswith("turns left > ")]
    self.assertEqual(prompts, ["10 turns left > "] * 4)

def test_valid_guess_is_recorded_and_counted_exactly_once(self):
    game = self.make()
    play(game, ["1111", "q"])
    self.assertEqual(game.history, [("1111", 1, 0)])
    self.assertEqual(game.turns, 9)

def test_each_accepted_guess_produces_exactly_one_new_history_row(self):
    game = self.make()
    out, _ = play(game, ["1111", "oops", "2222", "12", "3333", "q"])
    tables = [i for i, line in enumerate(out) if line.rstrip().startswith("#")]
    self.assertEqual(len(tables), 3) # one table per accepted guess
    self.assertEqual(len(game.history), 3)
    self.assertEqual(game.turns, 7)

def test_invalid_input_before_winning_guess(self):
    game = self.make()
    play(game, ["12", "9999", "", "1234"])
    self.assertEqual(game.state, WON)
    self.assertEqual(game.history, [("1234", 4, 0)])
    self.assertEqual(game.turns, 9)

def test_invalid_input_on_final_turn_does_not_end_the_game(self):
    game = self.make(max_turns=1)
    play(game, ["bad", "99", "1234"])
    self.assertEqual((game.state, game.turns), (WON, 0))
    game = self.make(max_turns=1)
    out, _ = play(game, ["bad", "1111"])
    self.assertEqual((game.state, game.turns), (LOST, 0))

def test_game_over_takes_precedence_over_validation(self):
    game = self.make()
    play(game, ["1234"])
    before = self.snapshot(game)
    for raw in ("9999", "", "1111"):
        with self.assertRaises(GameOver):
            game.submit(raw)
    self.assertEqual(self.snapshot(game), before)

def test_input_is_trimmed_but_not_otherwise_altered(self):
    game = self.make()
    play(game, [" 1111 ", "q"])
    self.assertEqual(game.history, [("1111", 1, 0)])

def test_quit_words(self):
    for word in ("q", "Q", "quit", "QUIT", " q "):
        with self.subTest(word=word):
            game = self.make()
            play(game, ["1111", word])
            self.assertEqual((game.state, game.turns, len(game.history)), (QUIT, 9, 1))

def test_end_of_input_and_ctrl_c_quit_cleanly(self):
    for error in (EOFError, KeyboardInterrupt):
        with self.subTest(error=error.__name__):
            game, out = self.make(), []
            feed = iter(["1111"])

            def read(prompt):
                try:
                    return next(feed)
                except StopIteration:
                    raise error

            game.run(read=read, write=out.append)
            self.assertEqual((game.state, game.turns, len(game.history)), (QUIT, 9, 1))

```

```

def test_menu_end_of_input_returns_none(self):
    def read(prompt):
        raise EOFError

    self.assertIsNone(choose_difficulty(read=read, write=lambda line: None))

def test_history_table_is_readable(self):
    game = self.make()
    game.submit("4413"[:-1])      # 3144 vs 1234
    game.submit("1243")
    game.submit("1234")
    self.assertEqual(game.history_lines(), [
        " # Guess Exact Partial",
        " 1 3 1 4 4      0      3",
        " 2 1 2 4 3      2      2",
        " 3 1 2 3 4      4      0",
    ])

def test_history_table_is_empty_before_first_guess_and_scales_with_length(self):
    game = self.make()
    self.assertEqual(game.history_lines(), [" # Guess Exact Partial"])
    hard = Mastermind(HARD, code=list("12345"))
    hard.submit("54321")
    header, row = hard.history_lines()
    self.assertEqual(len(header), len(row))
    self.assertIn("5 4 3 2 1", row)

def test_validation_follows_each_difficulty(self):
    for level in DIFFICULTIES:
        game = Mastermind(level)
        with self.subTest(level=level.name):
            for raw in ("", "1" * (level.length - 1), "1" * (level.length + 1),
                        "1" * (level.length - 1) + str(level.symbols + 1)):
                with self.assertRaises(InvalidGuess):
                    game.submit(raw)
            self.assertEqual((game.turns, game.history), (level.max_turns, []))
            game.submit(level.alphabet[-1] * level.length)
            self.assertEqual((game.turns, len(game.history)), (level.max_turns - 1, 1))

if __name__ == "__main__":
    open(p, "w").write(s)
    EOF
    python3 -m unittest discover -s tests 2>&1 | tail -30

```

TOOL RESULT 2026-10-07 16:53:12Z

```

- [' # Guess      Exact  Partial',
?                               --

+ [' # Guess  Exact  Partial',
- ' 1 3 1 4 4      1      2',
+ ' 1 3 1 4 4      0      3',
  ' 2 1 2 4 3      2      2',
  ' 3 1 2 3 4      4      0']

=====
FAIL: test_invalid_input_in_run_spends_no_turn_and_prints_no_feedback
(test_mastermind.ValidationTests.test_invalid_input_in_run_spends_no_turn_and_prints_no_feedback)
-----
Traceback (most recent call last):
  File "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind/tests/test_mastermind.py", line 353, in
test_invalid_input_in_run_spends_no_turn_and_prints_no_feedback
    self.assertEqual(unread, [])
    ~~~~~^~~~~~
AssertionError: Lists differ: ['q'] != []

First list contains 1 additional elements.
First extra element 0:
'q'

- ['q']
+ []

-----
Ran 53 tests in 0.072s

FAILED (failures=3)

```

TOOL CALL 2026-10-07 16:53:19Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && python3 -m unittest discover -s tests 2>&1 | grep -E
"^(FAIL|ERROR)"; python3 -c "
from game import *
g=Mastermind(Difficulty('t',4,6,10),code=list('1234'))
for x in ('3144','1243','1234'): g.submit(x)
print('\n'.join(g.history_lines()))"

```

TOOL RESULT 2026-10-07 16:53:19Z

```

FAIL: test_history_table_is_empty_before_first_guess_and_scales_with_length
(test_mastermind.ValidationTests.test_history_table_is_empty_before_first_guess_and_scales_with_length)
FAIL: test_history_table_is_readable (test_mastermind.ValidationTests.test_history_table_is_readable)
FAIL: test_invalid_input_in_run_spends_no_turn_and_prints_no_feedback
(test_mastermind.ValidationTests.test_invalid_input_in_run_spends_no_turn_and_prints_no_feedback)

```

CLAUDE 2026-10-07 16:53:22Z

TOOL CALL 2026-10-07 16:53:24Z

TOOL RESULT 2026-10-07 16:53:25Z

CLAUDE 2026-10-07 16:53:28Z

TOOL CALL 2026-10-07 16:53:28Z

TOOL RESULT 2026-10-07 16:53:28Z

Lab 4 chat history - page 28

```

44         if choice in (str(number), level.name):
45             return level
46         write(f"Please choose 1-{len(DIFFICULTIES)} or a difficulty name.")
47
48
49 class GameOver(Exception):
50     """Raised when a guess is submitted after the game has already ended."""
51
52
53 class Mastermind:
54     def __init__(self, difficulty=MEDIUM, code=None):
55         self.difficulty = difficulty
56         self.code = code or [str(random.randint(1, difficulty.symbols))
57                               for _ in range(difficulty.length)]
58         self.history = []
59         self.turns = difficulty.max_turns
60         self.state = PLAYING
61
62     @property
63     def finished(self):
64         return self.state != PLAYING
65
66     def submit(self, guess):
67         """Score one accepted guess and advance the game; return (exact, partial).
68
69         `guess` is a string (or sequence) of symbols. Raises GameOver once the
70         game has ended and InvalidGuess for a malformed guess; in both cases all
71         state is left untouched, so no turn is spent and no feedback is produced.
72         """
73         if self.finished:
74             raise GameOver(f"The game is already over ({self.state}).")
75         symbols = validate_guess("".join(guess), self.difficulty.length, self.difficulty.alphabet)
76         exact, partial = feedback(self.code, symbols)
77         self.history.append("".join(symbols), exact, partial)
78         self.turns -= 1
79         if exact == len(self.code):
80             self.state = WON
81         elif self.turns == 0:
82             self.state = LOST
83         return exact, partial
84
85     def quit(self):
86         if not self.finished:
87             self.state = QUIT
88
89     def history_lines(self):
90         """The guess history as a table, one row per accepted guess."""
91         width = max(5, 2 * self.difficulty.length - 1)
92         lines = [f"{'#':>3} {'Guess':<{width}} {'Exact':>5} {'Partial':>7}"]
93         for number, (guess, exact, partial) in enumerate(self.history, 1):
94             lines.append(f"{number:>3} {' '.join(guess):<{width}} {exact:>5} {partial:>7}")
95         return lines
96
97     def run(self, read=input, write=print):
98         level = self.difficulty
99         write(f"Mastermind ({level.name}) - enter {level.length} digits "
100              f"from 1 to {level.symbols}, or q to quit.")
101         while not self.finished:
102             try:
103                 raw = read(f"{self.turns} turns left > ").strip()
104             except (EOFError, KeyboardInterrupt):
105                 write("")
106                 raw = "q"
107             if raw.lower() in QUIT_WORDS:
108                 self.quit()
109                 break
110             try:
111                 self.submit(raw)
112             except InvalidGuess as problem:
113                 write(str(problem))
114                 continue
115             for line in self.history_lines():
116                 write(line)
117             self.announce_result(write)
118
119     def announce_result(self, write=print):
120         code = "".join(self.code)
121         guesses = len(self.history)
122         if self.state == WON:
123             write(f"Cracked the code in {guesses} guess{'es' if guesses != 1 else ''}!")
124         elif self.state == LOST:
125             write(f"Out of turns - you lose. The code was {code}")
126         elif self.state == QUIT:
127             write(f"You quit. The code was {code}")
128

```

TOOL CALL 2026-10-07 16:53:35Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && python3 - <<'EOF'
g=open("game.py").read()
blk='''class GameOver(Exception):
    """Raised when a guess is submitted after the game has already ended."""
'''
g=g.replace(blk,"",1)

```

```

g=g.replace(''QUIT_WORDS = ("q", "quit")

'', ''QUIT_WORDS = ("q", "quit")

''+blk.rstrip("\n")+"\n\n",1)
open("game.py","w").write(g)
EOF
sed -n 1,16p game.py; rm -rf __pycache__ tests/__pycache__; export PYTHONDONTWRITEBYTECODE=1; python3 -m unittest
discover -s tests 2>&1 | tail -3; python3 tests/test_mastermind.py 2>&1 | tail -2; git status --short; git diff --stat

```

TOOL RESULT 2026-10-07 16:53:35Z

```

import random
from dataclasses import dataclass
from logic import InvalidGuess, feedback, validate_guess

PLAYING, WON, LOST, QUIT = "playing", "won", "lost", "quit"
QUIT_WORDS = ("q", "quit")

class GameOver(Exception):
    """Raised when a guess is submitted after the game has already ended."""

@dataclass(frozen=True)
class Difficulty:
    name: str
    length: int # positions in the secret code
Ran 53 tests in 0.073s

OK

OK
M game.py
M logic.py
M tests/test_mastermind.py
Lab4/mastermind/game.py | 57 ++++++---
Lab4/mastermind/logic.py | 22 +++
Lab4/mastermind/tests/test_mastermind.py | 171 ++++++-----
3 files changed, 232 insertions(+), 18 deletions(-)

```

TOOL CALL 2026-10-07 16:53:38Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && git add game.py logic.py tests/test_mastermind.py &&
git commit -q -m "Lab 4: Harden input and history

Add validate_guess()/InvalidGuess in logic.py and have Mastermind.submit()
validate before touching any state, so malformed guesses (empty, wrong
length, invalid or look-alike symbols) never spend a turn, enter the history
or produce feedback. Print a readable per-guess history table, accept q/quit
(and Ctrl-D/Ctrl-C) as quit, and show specific error messages.

Tests cover malformed input of many kinds in every mode, repeated invalid
input, invalid input before a winning guess and on the final turn, and
exactly-once recording/turn/feedback for valid guesses.

Co-Authored-By: Claude Sonnet 5.5 <noreply@anthropic.com>" && git log --oneline | head -6

```

TOOL RESULT 2026-10-07 16:53:39Z

```

ccb9632 Lab 4: Harden input and history
0d6561f Lab 4: Add difficulty modes
a770f86 Lab 4: Complete game lifecycle
3b1a33a Lab 4: Fix duplicate-aware feedback
33abfde Lab 4: Add Mastermind baseline
46563d3 Add Lab 3: Component Modelling & Architectural Pattern Selection

```

TOOL CALL 2026-10-07 16:53:53Z

```

[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && python3 -
<<'EOF'
s=open("demo.py").read()
s=s.replace("    return seed, acts, pre",'    if label == "after":
    seed = 0
    pre.append(f"{GREEN}AFTER{RST} - final game (seed 0 -> secret 4413, shown for the demo only)")
    pre.append(f"{CYAN}Expect: bad input rejected without using a turn | 4444 -> Exact 2, Partial 0 | win ends the
game{RST}")
    type_("medium", 0.8)
    type_("12", 2.0)
    type_("4444", 3.0)
    type_("3414", 4.4)
    type_("4413", 5.8)
    say(f"{GREEN}FIXED: duplicate-aware feedback, difficulty menu, input validation, history table, win/loss
handling{RST}", 8.0)
    return seed, acts, pre')
open("demo.py","w").write(s)
EOF
S=$PWD; GD="/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind"
osascript -e 'tell application "Terminal" to do script "PS1=""'"$ "'"; clear" in front window' >/dev/null; sleep 1
rm -f after_raw.mov frame *.png
screencapture -v -V 10 -17231 after_raw.mov &
sleep 0.6

```

```
osascript -e "tell application \"Terminal\" to do script \"python3 $$demo.py after '$GD'\" in front window"
>/dev/null
wait; ls -la after_raw.mov; ./vinfo after_raw.mov 2.5 5.0 9.5
```

TOOL RESULT 2026-10-07 16:54:05Z

```
-rw-----@ 1 aadi  staff  2888260 Oct  7 22:24 after_raw.mov
duration: 10.02 s
frame_2.5.png 1692x1012
frame_5.0.png 1692x1012
frame_9.5.png 1692x1012
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL CALL 2026-10-07 16:54:07Z

```
[Read]
{
  "file_path":
"/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/frame_9.5.png"
}
```

TOOL RESULT 2026-10-07 16:54:07Z

[image omitted]

TOOL CALL 2026-10-07 16:54:08Z

```
[Read]
{
  "file_path":
"/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/frame_2.5.png"
}
```

TOOL RESULT 2026-10-07 16:54:08Z

[image omitted]

TOOL CALL 2026-10-07 16:54:13Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && rm -f
after_10s.mp4 && avconvert --preset PresetHighestQuality --source after_raw.mov --output after_10s.mp4 2>&1 | tail -1;
ls -la after_10s.mp4 keep/before_10s.mp4; ./vinfo after_10s.mp4; ./vinfo keep/before_10s.mp4; grep -c -a avc1
after_10s.mp4; cp after_10s.mp4 keep/; rm -f frame_*.png
```

TOOL RESULT 2026-10-07 16:54:15Z

```
-rw-r--r--@ 1 aadi  wheel  2887900 Oct  7 22:24 after_10s.mp4
-rw-r--r--@ 1 aadi  wheel  1374303 Oct  7 22:19 keep/before_10s.mp4
duration: 10.02 s
duration: 10.01 s
1
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

CLAUDE 2026-10-07 16:54:17Z

Both videos are done and verified (10.01 s and 10.02 s). Next I'm looking at how to export this session's chat history for the PDF and link.

TOOL CALL 2026-10-07 16:54:18Z

```
[ToolSearch]
{
  "query": "select:mcp_ccd_session_mgmt__export_transcript,mcp_ccd_session_mgmt__get_session,mcp_ccd_session_mgmt__l
ist_sessions",
  "max_results": 5
}
```

TOOL RESULT 2026-10-07 16:54:18Z

```
[tool_reference omitted]
[tool_reference omitted]
[tool_reference omitted]
```

CLAUDE 2026-10-07 16:54:22Z

First the session metadata, to see whether a shareable link exists.

TOOL CALL 2026-10-07 16:54:22Z

```
[mcp_ccd_session_mgmt__get_session]
{
  "session_id": "self"
}
```

TOOL RESULT 2026-10-07 16:54:22Z

```
{
  "sessionId": "local_804df02a-218d-4259-970a-fa0ebdf64c7a",
  "title": "Lab 4 Mastermind implementation",
  "cwd": "/Users/aadi/Desktop/SE lab 4",
  "isArchived": false,
  "isRunning": true,
  "lastActivityAt": "2026-10-07T16:54:22.796Z",
  "createdAt": "2026-10-07T16:46:12.223Z",
  "model": "claude-sonnet-5-5",
  "originCwd": "/Users/aadi/Desktop/SE lab 4",
  "isRemote": false,
  "effort": "high",
  "permissionMode": "auto",
```

```

"fastMode": {
  "state": "off",
  "disabledReason": "sdk_opt_in_required"
},
"outputStyle": "default",
"group": null,
"link": "claude://claude.ai/epitaxy/local_804df02a-218d-4259-970a-fa0ebdf64c7a",
"remoteControlActive": false,
"remoteControlState": "off"
}

```

TOOL CALL 2026-10-07 16:54:31Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && cat >> README.md <<'EOF'

```

Lab 4 implementation notes

Everything above is the original assignment text, kept unchanged. This section describes what was added for Lab 4.

How to play

```

```bash
python3 main.py
```

```

Pick a difficulty (by number or name), then enter guesses made of digits. `q` (or `quit`) ends the game. After every accepted guess the full history is shown:

```

```text
Guess Exact Partial
1 4 4 4 4 2 0
2 3 4 1 4 2 2
```

```

```

| Difficulty | Code length | Symbols | Guesses |
|---|---|---|---|
| easy      | 3 | 1-4 | 12 |
| medium (original game) | 4 | 1-6 | 10 |
| hard      | 5 | 1-8 | 8 |

```

What changed

- `logic.py` - `feedback()` is duplicate-aware: exact matches are resolved first, then only the unmatched positions are paired by symbol, so each code position counts at most once.
- `validate\_guess()` / `InvalidGuess` reject malformed guesses with a specific message.
- `game.py` - `Difficulty` configuration and start-up menu; an explicit game state
- (`playing` / `won` / `lost` / `quit`); `Mastermind.submit()` validates, scores and records exactly one guess and raises `GameOver` (changing nothing) once the game has ended; history table.
- `main.py` - asks for a difficulty, then runs the game.
- `tests/test\_mastermind.py` - standard-library `unittest` suite.

Running the tests

```

```bash
python3 -m unittest discover -s tests -v
```

```

State is kept in memory only; there are no third-party dependencies and nothing is written to disk.
 EOF
 tail -5 README.md; git diff --stat

TOOL RESULT 2026-10-07 16:54:31Z

```

```bash
python3 -m unittest discover -s tests -v
```

```

State is kept in memory only; there are no third-party dependencies and nothing is written to disk.
 Lab4/mastermind/README.md | 47 +++
 1 file changed, 47 insertions(+)

CLAUDE 2026-10-07 16:54:35Z

Next I'm exporting the transcript once now to see its format and build the PDF converter. I'll re-export at the very end so the final PDF is as complete as possible.

TOOL CALL 2026-10-07 16:54:35Z

```

[mcp__ccd_session_mgmt__export_transcript]
{
  "session_id": "self"
}

```

TOOL RESULT 2026-10-07 16:54:36Z

Exported this session's transcript "Lab 4 Mastermind implementation" to
 /Users/aadi/Downloads/session-export-1791392075826.zip (9 MB, 9013648 bytes). The zip holds the conversation transcript, its subagent transcripts and session metadata, not the app's logs; nothing was uploaded.

TOOL CALL 2026-10-07 16:54:38Z

```

[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && rm -rf

```

```
exp && mkdir exp && cd exp && unzip -q /Users/aadi/Downloads/session-export-1791392075826.zip && find . -type f | head
-20 && ls -la; for f in $(find . -type f | head -5); do echo "== $f"; head -c 600 "$f"; echo; done
```

TOOL RESULT 2026-10-07 16:54:39Z

```
./transcript.jsonl
./metadata.json
./local-session-state.json
./8a63cd23-403f-4a5f-a5d9-55e5f3137cf9.jsonl
./8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/custom-title.json
./8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/tool-results/pdf-2e20f18a-0f20-481d-ba34-398208fa4443/page-1.jpg
total 27976
drwxr-xr-x@ 7 aadi wheel 224 Oct 7 22:24 .
drwx-----@ 16 aadi wheel 512 Oct 7 22:24 ..
drwxr-xr-x@ 4 aadi wheel 128 Oct 7 22:24 8a63cd23-403f-4a5f-a5d9-55e5f3137cf9
-rw-r--r--@ 1 aadi wheel 6993594 Oct 7 22:24 8a63cd23-403f-4a5f-a5d9-55e5f3137cf9.jsonl
-rw-r--r--@ 1 aadi wheel 824 Oct 7 22:24 local-session-state.json
-rw-r--r--@ 1 aadi wheel 326995 Oct 7 22:24 metadata.json
-rw-r--r--@ 1 aadi wheel 6993594 Oct 7 22:24 transcript.jsonl
== ./transcript.jsonl
{"type":"queue-operation","operation":"enqueue","timestamp":"2026-10-07T16:46:14.000Z","sessionId":"8a63cd23-403f-4a5f-a5d9-55e5f3137cf9","content":"\n\n<pasted_content id=\"5177\">\nYou are acting as my autonomous Software Engineering Lab 4 implementation agent.\nYour job is to complete the entire Lab 4 assignment end-to-end on my computer, including inspecting the assigned repository, reproducing the original bug, recording the required before/after gameplay videos, implementing every required task, testing everything, maintaining clean Git history with separate commits, exporting the comple
== ./metadata.json
{"sessionId":"local_804df02a-218d-4259-970a-fa0ebdf64c7a","cliSessionId":"8a63cd23-403f-4a5f-a5d9-55e5f3137cf9","cwd":"/Users/aadi/Desktop/SE lab 4","originCwd":"/Users/aadi/Desktop/SE lab 4","gitAnchorsFolderRealpath":"/Users/aadi/Desktop/SE lab 4","lastFocusedAt":1791391572340,"createdAt":1791391572223,"lastActivityAt":1791391576798,"model":"claude-sonnet-5-5","effort":"high","isArchived":false,"title":"Lab 4 Mastermind implementation","titleSource":"auto","permissionMode":"auto","enabledMcpTools":{"80473450-eb87-474e-aed0-c7b572a37bd9: notion-create-pages-e3e17fe3ed552b9bf6d41e682a3ff41a":true
== ./local-session-state.json
{"capturedAt":"2026-10-07T16:54:35.490Z","sessionId":"local_804df02a-218d-4259-970a-fa0ebdf64c7a","cliSessionId":"8a63cd23-403f-4a5f-a5d9-55e5f3137cf9","sidebarShownState":"running","isRunning":true,"hasPendingPermission":false,"hasLiveWorkflows":false,"hasBackgroundWork":false,"hasBackgroundActivity":false,"activeBackgroundTasks":null,"pendingEchoUuids":null,"pendingEchoAgesMs":null,"awaitingTurnResult":null,"awaitingTurnResultAgesMs":null,"cliLastTurnMessageWasResult":false,"cliAtTurnBoundaryHint":false,"interruptResultPendingSince":null,"interruptResultPendingCycleArmed":null,"cliP
rovablyIdle
== ./8a63cd23-403f-4a5f-a5d9-55e5f3137cf9.jsonl
{"type":"queue-operation","operation":"enqueue","timestamp":"2026-10-07T16:46:14.000Z","sessionId":"8a63cd23-403f-4a5f-a5d9-55e5f3137cf9","content":"\n\n<pasted_content id=\"5177\">\nYou are acting as my autonomous Software Engineering Lab 4 implementation agent.\nYour job is to complete the entire Lab 4 assignment end-to-end on my computer, including inspecting the assigned repository, reproducing the original bug, recording the required before/after gameplay videos, implementing every required task, testing everything, maintaining clean Git history with separate commits, exporting the comple
== ./8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/custom-title.json
{"customTitle":"Lab 4 Mastermind implementation"}
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

CLAUDE 2026-10-07 16:54:42Z

The export is a JSONL transcript (about 7 MB, mostly embedded screenshots). Let me look at its structure and check which PDF tooling is available.

TOOL CALL 2026-10-07 16:54:45Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/exp && ls
"/Applications" | grep -i -E "chrome|chromium|edge|brave|firefox"; python3 -c "import reportlab; print('reportlab', reportlab.Version)" 2>&1 | tail -1; python3 -c "import fitz" 2>&1 | tail -1; which cupsfilter pandoc wkhtmltopdf; python3 - <<'EOF'
import json, collections
c = collections.Counter(); first = {}
with open("transcript.jsonl") as f:
    for i, line in enumerate(f):
        o = json.loads(line)
        t = o.get("type"); c[t] += 1
        first.setdefault(t, (i, list(o.keys())))
print(c)
for t, v in first.items(): print(t, v)
EOF
```

TOOL RESULT 2026-10-07 16:54:45Z

```
Brave Browser.app
Google Chrome.app
reportlab 5.0.1
ModuleNotFoundError: No module named 'fitz'
/usr/sbin/cupsfilter
pandoc not found
wkhtmltopdf not found
Counter({'assistant': 121, 'attachment': 73, 'user': 55, 'custom-title': 23, 'last-prompt': 22, 'agent-name': 22, 'atis-latch': 22, 'file-history-delta': 3, 'queue-operation': 2, 'file-history-snapshot': 1})
queue-operation (0, [{'type': 'operation', 'timestamp': 'sessionId', 'content'}])
user (2, [{'parentUuid', 'isSidechain', 'promptId', 'type', 'message', 'uuid', 'timestamp', 'permissionMode', 'origin', 'promptSource', 'turnOrigin', 'turnPosition', 'userType', 'entrypoint', 'cwd', 'sessionId', 'version', 'gitBranch'}])
attachment (3, [{'parentUuid', 'isSidechain', 'attachment', 'type', 'uuid', 'timestamp', 'rendered', 'renderedRole', 'userType', 'entrypoint', 'cwd', 'sessionId', 'version', 'gitBranch'}])
last-prompt (10, [{'type', 'lastPrompt', 'leafUuid', 'sessionId'}])
custom-title (11, [{'type', 'customTitle', 'sessionId'}])
agent-name (12, [{'type', 'agentName', 'sessionId'}])
file-history-snapshot (14, [{'type', 'messageId', 'snapshot', 'isSnapshotUpdate'}])
atis-latch (15, [{'type', 'atis', 'sessionId'}])
assistant (21, [{'parentUuid', 'isSidechain', 'message', 'thinkingDurationMs', 'serverClassifierRequest',
```

```
'apiBlockIndex', 'requestId', 'type', 'uuid', 'timestamp', 'advisorModel', 'effort', 'perTurnEffort', 'userType',
'entrypoint', 'cwd', 'sessionId', 'version', 'gitBranch'])
file-history-delta (164, ['type', 'messageId', 'snapshotMessageId', 'trackingPath', 'backup', 'timestamp'])
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL CALL 2026-10-07 16:55:02Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && python3 -
<<'EOF'
import json
# inspect block shapes
seen = {}
with open("exp/transcript.jsonl") as f:
    for line in f:
        o = json.loads(line)
        if o.get("type") in ("user", "assistant"):
            m = o["message"]; c = m["content"]
            if isinstance(c, str):
                seen.setdefault((o["type"], "str"), c[:200]); continue
        for b in c:
            k = (o["type"], b.get("type"))
            if k not in seen: seen[k] = json.dumps(b)[:300]
for k,v in seen.items(): print(k, ">", v, "\n")
EOF
```

TOOL RESULT 2026-10-07 16:55:02Z

```
('user', 'str') =>

<pasted_content id="5177">
You are acting as my autonomous Software Engineering Lab 4 implementation agent.
Your job is to complete the entire Lab 4 assignment end-to-end on my computer, including i

('assistant', 'thinking') => {"type": "thinking", "thinking": "", "signature": "CAQShgYKEAgSGAI4AUIIdGhpbmtpbmcSDL5pFm
UJ8ZHQwL6xdhoMHIwuUoxblyJBoCFDIjD/gcDzHKoRCrI49/Qb+O/ETTOTxDlJMr7P8dkyGMwpvkZDPp2gC4TDXqqltFW+CGkqowX7aWSF5Mxqf2bMsvlv
AxfVXV6fkoQRP2ofZe6xAhsfMxUVG34L3lv1NRYT+Ia5/xEJoic9mqFIuN6mrgHkUfzdk5JorzvTpVy2y5lRb+AhOsuAs

('assistant', 'text') => {"type": "text", "text": "I'll start by checking the tooling and getting both repositories."}

('assistant', 'tool_use') => {"type": "tool_use", "id": "toolu_01LK6rQB87fYdB86QaRNYvzY", "name": "Bash", "input":
{"command": "cd \"~/Users/aadi/Desktop/SE lab 4\" && ls -la && which git gh ffmpeg ffprobe python3 screencapture
asciinema; python3 --version; git --version; gh auth status 2>&1 | head -5; git config --global user.n

('user', 'tool_result') => {"tool_use_id": "toolu_01LK6rQB87fYdB86QaRNYvzY", "type": "tool_result", "content": "total
144ndrwxr-xr-x@ 3 aadi staff 96 Oct 7 22:09 .\ndrwx-----@ 112 aadi staff 3584 Oct 7 22:09
..\n-rw-r--r--@ 1 aadi staff 71595 Oct 7 10:49 Lab_4_VibeCoding_Student_handout.pdf\n/opt/homebrew/b
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL CALL 2026-10-07 16:55:21Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && cat >
mkpdf.py <<'EOF'
"""Convert an exported Claude Code session transcript (JSONL) to a readable PDF."""
import json, re, sys, textwrap
from reportlab.lib.pagesizes import A4
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.units import mm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import SimpleDocTemplate, Paragraph, Preformatted, Spacer, KeepTogether, HRFlowable

src, dst, link_note = sys.argv[1], sys.argv[2], sys.argv[3]
MONO = "/System/Library/Fonts/Supplemental/Courier New.ttf"
MONOB = "/System/Library/Fonts/Supplemental/Courier New Bold.ttf"
UNI = "/System/Library/Fonts/Supplemental/Arial Unicode.ttf"
pdfmetrics.registerFont(TTFont("Mono", MONO)); pdfmetrics.registerFont(TTFont("MonoB", MONOB))
pdfmetrics.registerFont(TTFont("Uni", UNI))
from reportlab.pdfbase.pdfmetrics import registerFontFamily
registerFontFamily("Mono", normal="Mono", bold="MonoB")
cov_mono = set(pdfmetrics.getFont("Mono").face.charToGlyph)
cov_uni = set(pdfmetrics.getFont("Uni").face.charToGlyph)

def clean(text, cov):
    text = text.replace("\t", " ").replace("\r", "")
    return "".join(ch if (ord(ch) in cov or ch == "\n") else "?" for ch in text)

SYS = re.compile(r"<system-reminder>.*?</system-reminder>", re.S)
MAXRES = 8000
stats = {"truncated": 0, "images": 0, "reminders": 0}

def block_text(c):
    """Flatten a tool_result content (str or list) to text."""
    if isinstance(c, str): return c
    out = []
    for b in c:
        if b.get("type") == "text": out.append(b["text"])
        elif b.get("type") == "image": out.append("[image omitted]"); stats["images"] += 1
        else: out.append(f"[{b.get('type')} omitted]")
    return "\n".join(out)

W = 118
def wrap(text, width=W):
```

```

lines = []
for ln in text.split("\n"):
    lines += textwrap.wrap(ln, width, replace_whitespace=False, drop_whitespace=False,
                           break_long_words=True, break_on_hyphens=False) or [""]
return "\n".join(lines)

body = ParagraphStyle("body", fontName="Mono", fontSize=7.2, leading=8.8)
tool = ParagraphStyle("tool", parent=body, fontSize=6.6, leading=8, backColor=colors.HexColor("#f3f4f6"),
                      borderPadding=3, leftIndent=6)
res = ParagraphStyle("res", parent=tool, backColor=colors.HexColor("#eef6ee"))
hdr = {r: ParagraphStyle("h" + r, fontName="MonoB", fontSize=8.5, leading=11, spaceBefore=7, spaceAfter=2,
                        textColor=c) for r, c in [("USER", colors.HexColor("#1d4ed8")),
                                                  ("CLAUDE", colors.HexColor("#b45309")),
                                                  ("TOOL CALL", colors.HexColor("#4b5563")),
                                                  ("TOOL RESULT", colors.HexColor("#166534"))]}

title = ParagraphStyle("t", fontName="MonoB", fontSize=13, leading=16, spaceAfter=4)

story = []
def add(role, text, style, stamp=""):
    text = text.rstrip()
    if not text: return
    story.append(Paragraph(f"{role} <font size=6.5 color='#6b7280'>{stamp}</font>", hdr[role]))
    story.append(Preformatted(wrap(clean(text, cov_mono), style)))

n_user = n_asst = n_tool = 0
with open(src) as f:
    for line in f:
        o = json.loads(line)
        t = o.get("type")
        if t not in ("user", "assistant"): continue
        stamp = o.get("timestamp", "")[:19].replace("T", " ") + "Z"
        c = o["message"]["content"]
        if isinstance(c, str): c = [{"type": "text", "text": c}]
        for b in c:
            bt = b.get("type")
            if bt == "text" and t == "user":
                txt, k = SYS.subn("", b["text"]); stats["reminders"] += k
                if txt.strip(): add("USER", txt, body, stamp); n_user += 1
            elif bt == "text":
                add("CLAUDE", b["text"], body, stamp); n_asst += 1
            elif bt == "tool_use":
                inp = b["input"]
                if b["name"] == "Bash" and "command" in inp:
                    desc = f"[Bash] {inp.get('description','')}\n$ {inp['command']}"
                else:
                    desc = f"[{b['name']}]\n" + json.dumps(inp, indent=1, ensure_ascii=False)
                if len(desc) > MAXRES: desc = desc[:MAXRES] + f"\n... [truncated, {len(desc)} chars total]";
                stats["truncated"] += 1
                add("TOOL CALL", desc, tool, stamp); n_tool += 1
            elif bt == "tool_result":
                txt = SYS.subn("", block_text(b.get("content", "")))
                if len(txt) > MAXRES: txt = txt[:MAXRES] + f"\n... [truncated, {len(txt)} chars total]";
                stats["truncated"] += 1
                add("TOOL RESULT", txt or "(no output)", res, stamp)
            elif bt == "image":
                stats["images"] += 1; add("USER", "[image omitted]", body, stamp)
                # 'thinking' blocks are empty/redacted in the export and are not rendered.

intro = [Paragraph("Lab 4 - VibeCoding: Mastermind - Complete LLM chat history", title),
        Paragraph(clean(
            f"Assistant: Claude (Claude Code, model claude-sonnet-5-5). Exported from the Claude Code desktop session
            "
            f"\nLab 4 Mastermind implementation\n" with the session-export feature, then rendered to PDF by a
            script.\n"
            f"Contents: {n_user} user messages, {n_asst} assistant messages, {n_tool} tool calls with their results,
            in order.\n"
            f"Omissions (rendering only): screenshots/images are shown as placeholders ({stats['images']}); tool
            outputs longer than "
            f"{MAXRES} characters are truncated ({stats['truncated']}); hidden model reasoning is not part of the
            export; "
            f"harness-injected <system-reminder> blocks ({stats['reminders']}) are removed, since they carry account
            metadata, not conversation.\n"
            f"Conversation link: {link_note}", cov_mono).replace("\n", "<br/>"),
        ParagraphStyle("i", fontName="Mono", fontSize=7.5, leading=10)),
        Spacer(1, 4), HRFflowable(width="100%", color=colors.grey)]

def footer(canvas, doc):
    canvas.setFont("Mono", 6.5); canvas.setFillColor(colors.grey)
    canvas.drawRightString(A4[0] - 12 * mm, 8 * mm, f"Lab 4 chat history - page {doc.page}")

doc = SimpleDocTemplate(dst, pagesize=A4, leftMargin=12*mm, rightMargin=12*mm, topMargin=12*mm, bottomMargin=14*mm,
                        title="Lab 4 - Complete LLM chat history", author="PES1UG24CS569")
doc.build(intro + story, onFirstPage=footer, onLaterPages=footer)
print(n_user, n_asst, n_tool, stats)
EOF
python3 mkpdf.py exp/transcript.jsonl test_chat.pdf
"claude://claude.ai/epitaxy/local_804df02a-218d-4259-970a-fa0ebdf64c7a (local app link)" && ls -la test_chat.pdf

```

TOOL RESULT 2026-10-07 16:55:22Z

```

2 17 53 {'truncated': 0, 'images': 10, 'reminders': 0}
-rw-r--r-- 1 aadi wheel 157177 Oct 7 22:25 test_chat.pdf
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4

```

CLAUDE 2026-10-07 16:55:27Z

The PDF builds (2 user messages, 17 assistant messages, 53 tool calls so far). Let me render a few pages to check how it looks.

TOOL CALL 2026-10-07 16:55:30Z

```
[Bash]
$ cd /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad && cat >
pdfpage.swift <<'EOF'
import PDFKit
import AppKit
let a = CommandLine.arguments
let doc = PDFDocument(url: URL(fileURLWithPath: a[1]))!
print("pages:", doc.pageCount)
for p in a.dropFirst(2) {
    let i = Int(p)! - 1
    guard let page = doc.page(at: i) else { continue }
    let r = page.bounds(for: .mediaBox)
    let img = page.thumbnail(of: CGSize(width: r.width * 2, height: r.height * 2), for: .mediaBox)
    let rep = NSBitmapImageRep(data: img.tiffRepresentation!)
    try! rep.representation(using: .png, properties: [:]).write(to: URL(fileURLWithPath: "page_\(p).png"))
}
EOF
swiftc -o pdfpage pdfpage.swift 2>&1 | grep -E "error" ; ./pdfpage test_chat.pdf 1 5
```

TOOL RESULT 2026-10-07 16:55:32Z

```
pages: 32
Shell cwd was reset to /Users/aadi/Desktop/SE lab 4
```

TOOL CALL 2026-10-07 16:55:33Z

```
[Read]
{
  "file_path":
  "/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/page_1.png"
}
```

TOOL RESULT 2026-10-07 16:55:33Z

```
[image omitted]
```

TOOL CALL 2026-10-07 16:55:34Z

```
[Read]
{
  "file_path":
  "/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/page_5.png"
}
```

TOOL RESULT 2026-10-07 16:55:34Z

```
[image omitted]
```

TOOL CALL 2026-10-07 16:55:41Z

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal" && git log --format='%h %s' | head -6 && sed -i '' 's/harness-injected
<system-reminder> blocks/harness-injected \&lt;system-reminder\&gt; blocks/'
/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/mkpdf.py && grep
-n "lt;system"
/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/mkpdf.py | head
-2
```

TOOL RESULT 2026-10-07 16:55:42Z

```
ccb9632 Lab 4: Harden input and history
0d6561f Lab 4: Add difficulty modes
a770f86 Lab 4: Complete game lifecycle
3b1a33a Lab 4: Fix duplicate-aware feedback
33abfde Lab 4: Add Mastermind baseline
46563d3 Add Lab 3: Component Modelling & Architectural Pattern Selection
105:      f"harness-injected &lt;system-reminder&gt; blocks ({stats['reminders']}) are removed, since they
carry account metadata, not conversation.\n"
```

TOOL CALL 2026-10-07 16:56:06Z

```
[Write]
{
  "file_path": "/Users/aadi/Desktop/SE lab 4/personal/Lab4/README.md",
  "content": "# Lab 4 - VibeCoding: Mastermind\n\n**Course:** Software Engineering (PES University, Dept. of CSE) ·
**Student:** PES1UG24CS569 · **Lab:** 4 - VibeCoding\n\n| |\n|---|---|\n| Assigned repository (Scenario 20 -
Mastermind) | https://github.com/SETAPESU26/20_mastermind |\n| Personal submission repository |
https://github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569 |\n\n## Objective\n\nUse an LLM coding assistant (Claude) to
take the assigned, deliberately broken Mastermind game,\nreproduce and fix its defect, and add the four features
listed in the assigned README - inspecting\nthe code first, testing every change, and keeping one Git commit per
task.\n\n**The defect.** `logic.feedback()` computed `partial` = (guessed symbols that appear anywhere in the code) -
`exact`,\nso one code position could be counted several times. With secret `4413` and guess `4444` it reported\n`Exact
2, Partial 2`; the correct answer is `Exact 2, Partial 0` (both 4s in the code are already used\nby the exact
matches). It was reproduced on the untouched code before any change was made (see `before_10s.mp4`).\n\n## Tasks
completed\n\n| Task | What was done | Where |\n|---|---|---|\n| **1 - Duplicate-aware feedback** | Two-pass algorithm:
exact matches are resolved first; only the remaining (unmatched) positions are then paired by symbol, so each code
position counts at most once. | `mastermind/logic.py` |\n| **2 - Complete game lifecycle** | Explicit state (`playing`
/ `won` / `lost` / `quit`); `Mastermind.submit()` scores and records one guess, enforces the guess limit, and raises
`GameOver` leaving all state untouched after the game ends; win / loss / quit messages; history preserved in every
outcome. | `mastermind/game.py` |\n| **3 - Difficulty modes** | `Difficulty` config + start-up menu: **easy** 3 digits
from 1-4, 12 guesses · **medium** 4 digits from 1-6, 10 guesses (the original game) · **hard** 5 digits from 1-8, 8
guesses. Code generation, validation, prompts and the guess limit all read from the chosen mode. |
```

```

'mastermind/game.py', 'mastermind/main.py' |\n| **4 - History and input robustness** | 'validate_guess()' /
'InvalidGuess' reject empty, wrong-length, out-of-range and look-alike (full-width / Arabic-Indic) digits with a
specific message; validation happens before any state change, so a bad guess never spends a turn, enters the history,
or produces feedback. A history table is shown after every accepted guess; 'q'/'quit', Ctrl-D and Ctrl-C quit cleanly.
| 'mastermind/logic.py', 'mastermind/game.py' |\n\n## Testing\n\nA standard-library 'unittest' suite (**53 tests**,
no third-party packages) lives
in\n[ 'mastermind/tests/test_mastermind.py' ] (mastermind/tests/test_mastermind.py).\n\n``bash\nncd
Lab4/mastermind\npython3 -m unittest discover -s tests -v\n```\n\nResult of the final run: 'Ran 53 tests ... OK'. Run
against the original 'logic.py', the Task 1 feedback tests fail\n(3,628 failing sub-tests), confirming they detect the
defect.\n\nRequired case | Covered by (test class → test) |\n|---|---|\n| Exact matches | 'FeedbackTests' →
'test_all_exact' |\n| Repeated symbols (guess only / code only / both) | 'FeedbackTests' →
'test_repeated_symbols_in_*', 'test_original_defect_case' |\n| No matches | 'FeedbackTests' → 'test_no_matches' |\n|
Mixed exact + partial | 'FeedbackTests' → 'test_mixed_exact_and_partial', 'test_exact_resolved_before_partial' |\n|
Independent check of the algorithm | 'FeedbackTests' → 'test_matches_independent_reference_exhaustively' (6,561
code/guess pairs vs. a counting formula) |\n| Every difficulty | 'DifficultyTests' → distinct config, valid generated
codes, validation, guess limit, win/loss and duplicate-aware feedback in **each** mode, menu |\n| Invalid / malformed
guesses | 'ValidationTests' → 16 malformed inputs (empty, blank, short, long, letters, 0, out of range, spaces,
punctuation, sign, decimal, tab, full-width and Arabic-Indic digits, newline) |\n| Invalid guesses spend no turn, add
no history, give no feedback | 'ValidationTests' → 'test_submit_rejects_malformed_guess_without_changing_state',
'test_invalid_input_in_run_spends_no_turn_and_prints_no_feedback',
'test_prompt_turn_counter_does_not_move_on_invalid_input' |\n| Valid guess recorded / counted / answered exactly once
| 'ValidationTests' → 'test_valid_guess_is_recorded_and_counted_exactly_once',
'test_each_accepted_guess_produces_exactly_one_new_history_row' |\n| First-turn win | 'LifecycleTests' →
'test_immediate_win' |\n| Last-turn (final-guess) win | 'LifecycleTests' →
'test_win_on_final_turn_is_a_win_not_a_loss'; 'DifficultyTests' → 'test_win_and_loss_in_every_mode' |\n| Loss on the
final allowed guess | 'LifecycleTests' → 'test_loss_after_all_guesses_used'; 'DifficultyTests' →
'test_guess_limit_matches_the_mode' |\n| Invalid input right before a winning guess / on the final turn |
'ValidationTests' → 'test_invalid_input_before_winning_guess',
'test_invalid_input_on_final_turn_does_not_end_the_game' |\n| Multiple / repeated invalid inputs | 'ValidationTests' →
'test_invalid_input_in_run_spends_no_turn_and_prints_no_feedback' |\n| Quitting | 'LifecycleTests' → 'test_quit_*';
'ValidationTests' → 'test_quit_words', 'test_end_of_input_and_ctrl_c_quit_cleanly' |\n| State / history unchanged
after the game ends | 'LifecycleTests' → 'test_state_frozen_after_win_loss_and_quit',
'test_run_reads_no_input_after_game_ends', 'test_history_is_intact_when_game_is_lost'; 'ValidationTests' →
'test_game_over_takes_precedence_over_validation' |\n| Readable history | 'ValidationTests' →
'test_history_table_is_readable', 'test_history_table_is_empty_before_first_guess_and_scales_with_length' |\n\n##
Deliverables\n\nDeliverable | File |\n|---|---|\n| Video **before** (10 s, original code, bug visible) |
[evidence/before_10s.mp4] (evidence/before_10s.mp4) |\n| Video **after** (10 s, final game) |
[evidence/after_10s.mp4] (evidence/after_10s.mp4) |\n| Complete LLM chat history (PDF) |
[evidence/LLM_Chat_History.pdf] (evidence/LLM_Chat_History.pdf) |\n| LLM chat link |
[evidence/LLM_Chat_Link.txt] (evidence/LLM_Chat_Link.txt) |\n| Updated source code | [ 'mastermind/' ] (mastermind/)
|\n\n**How the videos were made.** Both are real screen recordings (macOS 'screencapture', window-only, ~10 s, H.264
MP4) of the actual game\n\nrunning in a Terminal window. To make them repeatable, a small helper script (not part of the
submission) launched the game with\n\nrandom.seed(0) - which gives the secret '4413' - through 'runpy', without
modifying any game source, and typed the guesses with\n\nhuman-like timing. The secret and the expected results are
printed above the game by the helper for the viewer's benefit; the game's own\n\noutput is unmodified.\n\n\n## Running the
game\n\n``bash\nncd Lab4/mastermind\npython3 main.py\n```\n\nRequires Python 3.9+ and nothing else.\n\n\n## Constraints
verified\n\n- **No unnecessary external dependencies** - only the Python standard library ('random', 'collections',
'dataclasses'; 'unittest' for tests). 'requirements.txt' is unchanged.\n\n- **No persistence** - no files, CSV, JSON,
SQLite or database are read or written; all game state lives in memory.\n\n- **Original project structure and concept
retained** - same four files ('main.py', 'game.py', 'logic.py', 'requirements.txt'), still a terminal Mastermind; the
assigned 'README.md' text is preserved with a Lab 4 section appended.\n\n- Labs 1-3 are untouched; all Lab 4 work is
inside 'Lab4/'\n\n- Nothing was pushed to, and no pull request was opened against, 'SETAPESU26/20_mastermind'.\n\n\n##
Git commits\n\n# | Commit | Message |\n|---|---|---|\n| 1 | '33abfde' | Lab 4: Add Mastermind baseline |\n| 2 |
'3b1a33a' | Lab 4: Fix duplicate-aware feedback - **Task 1** |\n| 3 | 'a770f86' | Lab 4: Complete game lifecycle -
**Task 2** |\n| 4 | '0d6561f' | Lab 4: Add difficulty modes - **Task 3** |\n| 5 | 'ccb9632' | Lab 4: Harden input and
histo
... [truncated, 8873 chars total]

```

TOOL RESULT 2026-10-07 16:56:06Z

File created successfully at: /Users/aadi/Desktop/SE lab 4/personal/Lab4/README.md (file state is current in your context - no need to Read it back)

CLAUDE 2026-10-07 16:56:13Z

Before moving on, one honesty fix: I only ran the code on Python 3.14, so the README shouldn't claim more than that. Then I'll assemble the 'evidence/' folder.

TOOL CALL 2026-10-07 16:56:20Z

```

[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4" && python3 - <<'EOF'
p="README.md"; s=open(p).read()
s=s.replace("Requires Python 3.9+ and nothing else.", "Standard library only (the project's recommended Python is 3.9+;
it was developed and tested here with Python 3.14).")
open(p,"w").write(s)
EOF
S=/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad
mkdir -p evidence && cp $S/keep/before_10s.mp4 $S/keep/after_10s.mp4 evidence/ && cat > evidence/LLM_Chat_Link.txt
<<'EOF'
Lab 4 - Mastermind: LLM chat link
=====

```

Assistant used : Claude (Claude Code desktop app), model claude-sonnet-5-5
Session title : Lab 4 Mastermind implementation
Session id : local_804df02a-218d-4259-970a-fa0ebdf64c7a
Session link : claude://claude.ai/epitaxy/local_804df02a-218d-4259-970a-fa0ebdf64c7a

What this link is

This lab was done in a local Claude Code desktop session. The session has no public "share" page, so the only link the app provides is the deep link above. It opens the session only inside the Claude desktop app on the account/computer that ran it - it is NOT a public web URL. No public share link was generated or invented.
Complete chat history

The full conversation is provided as LLM_Chat_History.pdf in this folder. It was generated from the session's own transcript export (Claude Code "Export" -> transcript.jsonl) and contains the lab prompt, Claude's replies, and every tool call (commands run, files written) with its output, in order. Rendering notes: screenshots appear as placeholders, hidden model reasoning is not part of the export, and harness-injected system reminders (account metadata) were removed. The final packaging/push steps happen after the export was taken, so they are not in the PDF.

EOF

```
ls -la evidence; cd mastermind && git diff --no-index --stat /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream/requirements.txt requirements.txt && echo "requirements.txt identical to upstream"
```

TOOL RESULT 2026-10-07 16:56:20Z

```
total 8344
drwxr-xr-x@ 5 aadi  staff      160 Oct  7 22:26 .
drwxr-xr-x@ 5 aadi  staff      160 Oct  7 22:26 ..
-rw-r--r--@ 1 aadi  staff    1315 Oct  7 22:26 LLM_Chat_Link.txt
-rw-r--r--@ 1 aadi  staff  2887900 Oct  7 22:26 after_10s.mp4
-rw-r--r--@ 1 aadi  staff   1374303 Oct  7 22:26 before_10s.mp4
requirements.txt identical to upstream
```

CLAUDE 2026-10-07 16:56:22Z

Now the final quality pass: a clean-state test run, a normal unseeded game, a diff against upstream, and a check for stray files or dependencies.

TOOL CALL 2026-10-07 16:56:25Z

```
[Bash]
$ cd "/Users/aadi/Desktop/SE lab 4/personal/Lab4/mastermind" && export PYTHONDONTWRITEBYTECODE=1 && find . -name
__pycache__ | wc -l && python3 -m unittest discover -s tests 2>&1 | tail -4 && python3 -m py_compile main.py game.py
logic.py tests/test_mastermind.py && echo compiled-ok && rm -rf __pycache__ tests/__pycache__ && printf
'1\n123\n44\nq\n' | python3 main.py | tail -8; echo "--- imports:"; grep -hE "(import|from)" main.py game.py
logic.py; echo "--- file I/O / persistence:"; grep -nE "open\(|json|csv|sqlite|pickle|shelve|write\|.*\bfile" main.py
game.py logic.py || echo none; echo "--- upstream diff names:"; diff -rq
/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream . -x
.git -x tests -x __pycache__; cd ../../ && git status --short
```

TOOL RESULT 2026-10-07 16:56:26Z

1

Ran 53 tests in 0.073s

OK

compiled-ok

- 1) easy - 3 digits from 1-4, 12 guesses
- 2) medium - 4 digits from 1-6, 10 guesses
- 3) hard - 5 digits from 1-8, 8 guesses

Difficulty (number or name, q to quit) > Mastermind (easy) - enter 3 digits from 1 to 4, or q to quit.

12 turns left > # Guess Exact Partial

1 1 2 3 1 0

11 turns left > A guess needs exactly 3 digits - you entered 2.

11 turns left > You quit. The code was 343

--- imports:

from game import Mastermind, choose_difficulty

from collections import Counter

import random

from dataclasses import dataclass

from logic import InvalidGuess, feedback, validate_guess

--- file I/O / persistence:

none

--- upstream diff names:

Files /private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream/README.md and ./README.md differ

Files

/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream/game.py and ./game.py differ

Files

/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream/logic.py and ./logic.py differ

Files

/private/tmp/claude-501/-Users-aadi-Desktop-SE-lab-4/8a63cd23-403f-4a5f-a5d9-55e5f3137cf9/scratchpad/upstream/main.py and ./main.py differ

M Lab4/mastermind/README.md

?? Lab4/README.md

?? Lab4/evidence/